

Shahjalal University of Science and Technology

Department of Computer Science and Engineering



Sentimental Style Transfer in Bengali Text with Variational Autoencoder

SNEHA DEVI

Reg. No.: 2016331105

4th year, 1st Semester

MD. OMUR FARUQUE

Reg. No.: 2015331007

5th year, 1st Semester

Department of Computer Science and Engineering

Supervisor

SUMMIT HAQUE

Assistant Professor

Department of Computer Science and Engineering

8th July, 2021

Sentimental Style Transfer in Bengali Text with Variational Autoencoder



A Thesis submitted to the Department of Computer Science and Engineering, Shahjalal University of Science and Technology, in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

By

Sneha Devi

Reg. No.: 2016331105

4th year, 2nd Semester

Md. Omur Faruque

Reg. No.: 2015331007

5th year, 2nd Semester

Department of Computer Science and Engineering

Supervisor

SUMMIT HAQUE

Assistant Professor

Department of Computer Science and Engineering

8th July, 2021

Recommendation Letter from Thesis/Project Supervisor

The thesis entitled *Sentimental Style Transfer in Bengali Text with Variational Autoencoder* submitted by the students

1. Sneha Devi
2. Omur Faruque

is under my supervision. I, hereby, agree that the thesis/project can be submitted for examination.



Signature of the Supervisor:

Name of the Supervisor: Summit Haque

Date: 8th July, 2021

Certificate of Acceptance of the Thesis

The thesis entitled *Sentimental Style Transfer in Bengali Text with Variational Autoencoder* submitted by the students

1. Sneha Devi
2. Md. Omur Faruque

on 8th July, 2021, hereby, accepted as the partial fulfillment of the requirements for the award of their Bachelor Degrees.



Head of the Dept.

Dr. Mohammad Abdullah Al
Mumin

Professor and Head

Department of Computer
Science and Engineering

Chairman, Exam. Committee

Dr. Mohammad Abdullah Al
Mumin

Professor and Head

Department of Computer
Science and Engineering

Supervisor

Summit Haque

Assistant Professor

Department of Computer
Science and Engineering

Abstract

With the explosion of digital data and ever powerful computing power, machine learning architectures have surpassed human level performance in many different tasks. Working with textual data has become much effective and versatile than ever. In this vast area of textual data analysis, we made an attempt on looking into the style transfer of text. We narrowed down our objectives to focus only on positive-negative sentiments. While there are little room for improvements in most NLP tasks, probabilistic deep learning models are still largely unexplored and suits our purpose very well. Also, there has been no significant work on Bengali language in this region. We explored different discriminative and generative model architectures in this study. We have found that, traditional Seq2Seq models with recurrent neural networks find it harder than their capabilities when tackling this type of generative objectives. As a generative model for sentimental style transfer for Bengali, we initially proposed a Variational Autoencoder with attribute vector as a sentiment margin to direct the sentiment of a sentence to a desired target. We found this model to be vaguely defined when transferring sentiments. To remedy this problem, we propose our final model as a hybrid of VAE and GAN by adding a discriminator to the model which can guide the decoder properly while transferring sentiments.

Keywords: Variational Autoencoder, VAE, Generative Adversarial Networkm GAN, Probabilistic Model, Generative Model, Linguistic Style, Style Transfer, Textual Style Transfer, Sentiment Translation, Negative to Positive, Positive to Negative.

Acknowledgements

We would like to thank the Department of Computer Science and Engineering, Shahjalal University of Science and Technology, Sylhet 3114, Bangladesh, for supporting this research. We are very thankful to our honorable supervisor Summit Haque for his worthy supports and directions.

Contents

Abstract	I
Acknowledgements	II
Table of Contents	III
List of Figures	VII
1 Introduction	1
1.1 Style Transfer	1
1.1.1 Neural Style Transfer	2
1.1.2 Text Style Transfer	2
1.2 Different data pipelines	3
1.2.1 Parallel Data	3
1.2.2 Non-parallel Data	3
1.3 Uses of Style Transfer	3
1.3.1 Data Augmentation	4
1.3.2 Chat-bots	4
1.3.3 Poetry Generation	4
1.4 Motivation	4
1.5 Goal	5
2 Background Study	6
2.1 Artificial Neural Networks	7
2.1.1 Architecture	7
2.1.2 Forward Pass	10
2.1.3 Loss Function	11

2.1.4	Optimization	11
2.2	Recurrent Neural Network	12
2.2.1	Architecture	12
2.2.2	Forward Pass	13
2.2.3	Training	15
2.2.4	Drawbacks	15
2.3	Long Short Term Memory	16
2.3.1	Architecture	16
2.3.2	Improvements over Vanilla RNN	17
2.3.3	Drawbacks	18
2.4	Gated Recurrent Unit	19
2.5	Autoencoder	20
2.5.1	Architecture	20
2.5.2	Training	20
2.5.3	Use cases	21
2.6	Variational Autoencoders	22
2.6.1	Probabilistic Models	22
2.6.2	Architecture	22
2.7	Generative Adversarial Network	23
2.7.1	Architecture	24
2.7.2	Training	25
2.7.3	Variations and use cases	26
3	Literature Review	28
3.1	Relevant Works relating to our objective	28
3.1.1	Sequence to Better Sequence: Continuous Revision of Combinatorial Structures	28
3.1.2	Decomposing Textual Information For Style Transfer	29
3.1.3	Delete, Retrieve, Generate: A Simple Approach to Sentiment and Style Transfer	29
3.1.4	Sentiment Transfer using Seq2Seq Adversarial Autoencoders	29

3.1.5	Unpaired Sentiment-to-Sentiment Translation: A Cycled Reinforcement Learning Approach	30
3.1.6	Learning Sentiment Memories for Sentiment Modification without Parallel Data	30
3.1.7	Contextual Text Style Transfer	30
3.1.8	Domain Adaptive Text Style Transfer	31
3.1.9	IMaT: Unsupervised Text Attribute Transfer via Iterative Matching and Translation	31
3.1.10	Sentimental Style Transfer in Text with Multigenerative Variational Auto-Encoder	32
3.2	Other notable researches	32
4	Methodology	33
4.1	Exploring Candidate Models	33
4.1.1	Word Embeddings	33
4.1.2	Seq2Seq	36
4.1.3	Generative Models	36
4.2	Proposed Model	37
4.2.1	Difference between Autoencoder and VAE	37
4.2.2	Difficulties in Training Variational Autoencoders	38
4.2.3	Variational Bayes	39
4.2.4	Kullback Leibler Divergence	40
4.2.5	Evidence Lower Bound	41
4.2.6	Monte Carlo Estimate	41
4.2.7	Reparameterization Trick	42
4.2.8	Attribute Vector	42
4.2.9	Hybrid Structure with GAN	43
4.2.10	VAE Training	44
4.2.11	Generator Training	45
4.2.12	Discriminator Training	46
4.2.13	Challenge in attribute learning	46

4.2.14 Training Overview	47
5 Implementation and Results	48
5.1 Embedding Selection	48
5.2 Preprocessing	49
5.3 Datasets	50
5.4 Encoder	51
5.5 Generator	53
5.6 Discriminator	53
5.7 Training	54
5.8 Evaluation	55
6 Conclusion	59
References	60

List of Figures

1.1	Neural Style Transfer with images	2
2.1	Google Trend showing more attention toward deep learning in recent years	6
2.2	Anatomy of a Human Neuron	7
2.3	Overview of an Artificial Neuron	8
2.4	A Multilayer Perceptron Model	8
2.5	A Deep Neural Network	9
2.6	An RNN cell	12
2.7	Unrolled view of an RNN cell	13
2.8	An RNN cell overview	13
2.9	Recurrent Neural Network relations	14
2.10	An LSTM Cell	16
2.11	Unrolled view of an LSTM Cell	17
2.12	Overview of a GRU Cell	19
2.13	An Autoencoder with 3 dimensional latent space	20
2.14	A Variational Autoencoder with 2 dimensional latent space	23
2.15	Fake celebrity images generated by GAN	24
2.16	Generative Adversial Network	25
2.17	CycleGAN transforming a horse to a zebra	26
4.1	Continuous Bag of Words with 4 Context Words	34
4.2	Skipgram Embedding	35
4.3	Overview of a Sequence to Sequence Model	36

4.4	Difference of latent space between Autoencoder and VAE	37
4.5	Illustration of the VAE model	39
4.6	Updated VAE architecture with structured code $\mathbf{c}$	43
4.7	Hybrid Architecture with VAE and GAN	44
4.8	GAN part of the model	45
5.1	Encoder's structure	52
5.2	Decoder's structure	53
5.3	Discriminator's structure	54
5.4	Positive to Negative Style Transfer without discriminator	56
5.5	Negative to Positive Style Transfer without discriminator	56
5.6	Positive to Negative Style Transfer with discriminator	57
5.7	Negative to Positive Style Transfer with discriminator	57

Chapter 1

Introduction

In the field of Natural Language Processing, neural text generation models are becoming increasingly popular. Music lyrics generation, language translation, poem generation, and story completion are all excellent instances of Neural Text Generation. Linguistic style transfer also falls under this category and shows significant scopes of application. Language is the means by which humans communicate with one another. Language also gives you the freedom to express yourself in a multitude of ways. Teaching computers to grasp human natural language, let alone the type of writing people use, is tough. Despite the fact that Neural Style Transfer is very successful in computer vision domain, it has a lower success rate in Natural Language Processing, since text synthesis and manipulation are more difficult. The lack of sufficient parallel data is the most difficult problem to solve. As a result, deep generative models must be trained on a non-parallel corpus, which is more difficult. We give machines this challenging task and ask them to figure out how to adapt the style of human language.

1.1 Style Transfer

Style Transfer is an optimization approach that effectively transfers the style of an object by combining its contents with the style of another object. This object can be either image or text. In domains such as Natural Language Processing (NLP), Computer Vision and others, style transfer is a difficult subject. For a machine to manipulate the style, it must intuitively understand the fundamental meaning or context. When properly developed, a model gains the capacity to act as

an intelligent system, generating new content based on the input data.

1.1.1 Neural Style Transfer

Neural Style Transfer (NST) is a group of software algorithms that transform images or videos to mimic the appearance or visual style of a different image. Deep Neural Networks are used for image alteration in NST algorithms. An example of image style transfer is shown in **Figure 1.1**. Among these six images the first image is the original image and rest of the images are transformed images using five different image style on the original one. And the respective reference style images are also attached with these five transformed images on the left bottom corner.



Figure 1.1: Neural Style Transfer with images

1.1.2 Text Style Transfer

Style transfer has picked momentum in the field of Natural Language Processing in recent years. A few works somewhat linked to our domain include creating news titles, music lyrics generation, poetry generation [1] in a given style, and so on. However, style transfer in NLP continues to lag considerably in comparison to other fields, owing to the scarcity of parallel data. As a result, classical discriminative models are ineffective, and generative models that can be trained on non-parallel text are required.

Natural Language Generation, or NLG, is a challenging problem in Natural Language Process-

ing. Text Style Transfer is an important task in NLG systems that generates output by changing a few text qualities including emotions, politeness, positivism, negativism, comedy, and so on. In recent years, there has been a spike of interest in this subject, and significantly better results are being seen as a result of major improvements in deep neural network performance.

1.2 Different data pipelines

Depending on the structure of the corpora, multiple types of corpora can be employed in Natural Language Generative models. The corpus could consist of pairs of sentences written in two different languages or styles. It could also be a group of non-paired sentences. The corpus comprising paired sentences is, in essence, supervised data. The corpus of non-paired sentences, on the other hand, is unsupervised data.

1.2.1 Parallel Data

A parallel corpus is a collection of texts written in two different forms. The English to German language translation corpus, for example, contains sets of identical sentences in both English and German. These types of corpora can be utilized to create supervised models for translating from English to German and vice versa. In a supervised scenario, it is easy to predict the target data, but the irony is that parallel data is often difficult to collect for purposes like sentimental style transfer because they are often ambiguous and there can be thousands of different styles, each of which can be achieved in a variety of ways.

1.2.2 Non-parallel Data

The non-parallel corpus is the polar opposite of the parallel corpus, and there is plenty of these data available. The only issue with these data is that they are difficult to manipulate. Although, with the right skills and methodologies, these data can be put to good use.

1.3 Uses of Style Transfer

There can be many different scenarios where style transfer of texts can be very fruitful. We will mention just a interesting few here.

1.3.1 Data Augmentation

Well preprocessed and plentiful data acts like the skeleton of NLP models. It is quite difficult to augment textual data where more data for a model is needed or a biased dataset is available. Neural Style Transfer can be used as an interesting deep learning approach which can fulfill the need of data augmentation.

1.3.2 Chat-bots

An AI personal assistant or chat-bots improvised with Generative Style Transfer can be customized according to the specific demands. Every human has his own style of producing text and communicating with others. There are many famous writers like William Shakespeare and people like their writing all around the world. However, humans express their emotions in their language. On the other hand, interacting with the so called chat-bots are so emotionless that it feels like searching a query in the database. But the chat-bot that uses **Generative Style Transfer** can mimic the style of specific persons based on emails, chats and writing style data. For example, a chat-bot can be trained to mimic the famous writer, William Shakespeare.

1.3.3 Poetry Generation

Generation of poems are also possible using style transfer. A study showed successful generation of chinese poems by extracting data from news [1]. These type of models can be developed using generative models like Variational Autoencoders [2] Generative Adversarial Networks or GANs [3] or cyclic-GANs [4] etc. They can be trained from non-parallel corpus like the paper mentioned above.

1.4 Motivation

State of the art Natural Language Generation system are far away from human level performance unlike other regions like image generation. This was the main motivation for us to select our area of interest. From a vast and diverse field of NLG, we decided to explore one of the most interesting topic in this domain, to transfer style of a given sentence. It has some very useful use-cases, which we mentioned briefly above.

We looked for existing research and works on Bengali language in this field. Unfortunately, we found nearly no significant works for Bengali language that shows good metrics related to our objective. We found only one recent work that tries to tackle this problem in a generative way [5]. This made us motivated more as there are enough scopes for a novel work that can help others trying to work in this area.

Generative models like this in the field of Natural Language Processing is relatively new and shows a lot of potential and scope to be improved on. This was also motivating for us as there is a lot of scope for future works.

1.5 Goal

We see a lot of scope to work on our chosen objective. We also see a lot of potential use if we can contribute a good enough model. We plan to explore related models that fits our objectives well and plan to design a good enough model that can be used for Bengali language in various domains.

Chapter 2

Background Study

Machine Learning is not new. But the advances we are making are progressing at an exponential rate, leveraging our ever faster computing power, explosion of digital data and the speed of communication infrastructure. History of Artificial Intelligence can be traced back to 1950 when Alan Turing proposed the Turing Test [6]. In 1951, first AI based program was written. Since then, there has been a tremendous amount of research made in the field of machine learning. Subsequently, new theories, algorithms and research papers in this field grew exponentially. Although it is not a new field, model architectures and algorithms is still evolving very much as it is an active field of research even today.

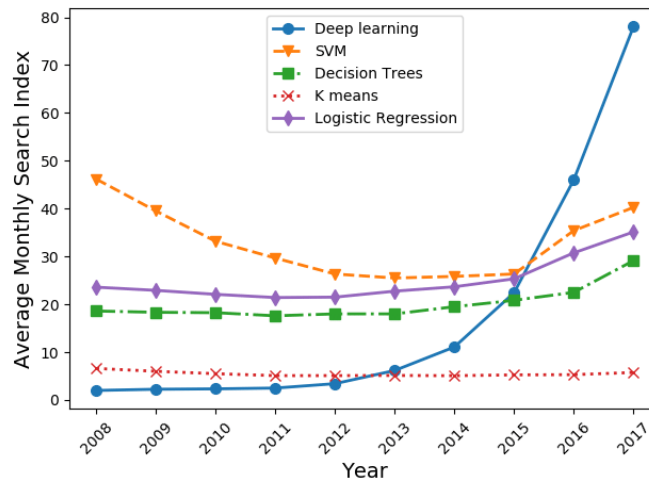


Figure 2.1: Google Trend showing more attention toward deep learning in recent years

Not long ago, machine learning architecture or models were entirely different for different tasks. People needed to use sophisticated architecture for specific problems. Today, almost all of the modern machine learning architectures involves deep learning. The surge in deep learning interests is shown in **Figure 2.1**. It shows the average number of monthly google search of different machine learning architecture. The reason behind this trend is that neural networks are very versatile. They can be modified in many different ways to make almost all kind of machine learning models. We will try to demonstrate few of those models briefly that helped making the compound model architecture for our work.

2.1 Artificial Neural Networks

2.1.1 Architecture

There is a reason why Artificial Neural Networks or ANN are called the way they are. Neural Networks are loosely based on how the human brain works. Human brain and our nervous system are made of small cells called neurons. These neurons are connected to numerous others and they take incoming signals from other neurons through some fine structures called dendrites. Based on the incoming signals, a neuron then passes different level of signals forward to other neurons through axon, which can split into thousands of branches. Our brain processes everything with these signals or spikes of electrical activity inside these neurons. Whether it is walking, seeing, hearing or singing, our brain functions in this same manner. **Figure 2.2** below illustrates the structure of a single neuron.

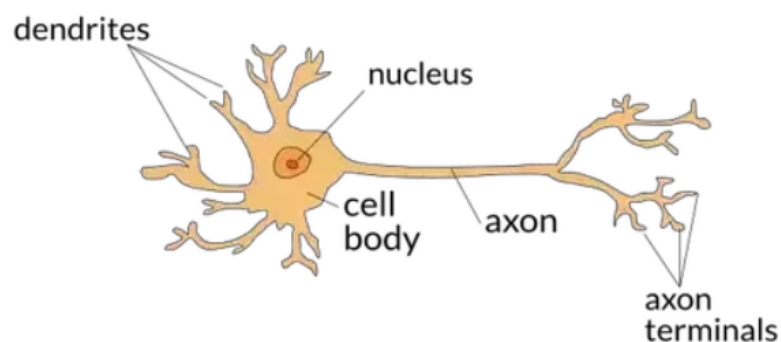


Figure 2.2: Anatomy of a Human Neuron

Artificial Neural Network also consists of computation cells called neurons. They try to somewhat mimic the way a biological neuron works. They take outputs from other incoming neuron activations and computes a value called logit (z) with the linear combination of them added with a bias value. The logit is then passed through a function (f) called activation function to calculate the activation for that neuron. The activation is then forwarded to other connected neurons. These similarities in artificial neuron is illustrated in **Figure 2.3**.

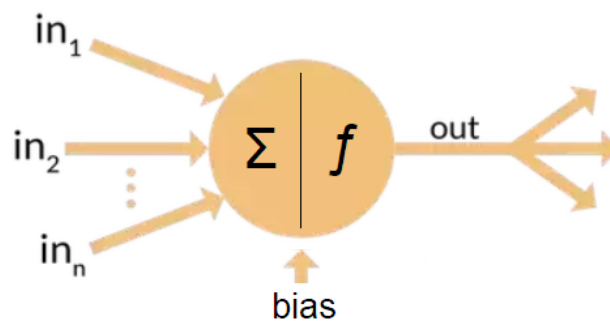


Figure 2.3: Overview of an Artificial Neuron

ANN leverages the use of similar behaving neurons collectively in layers. Multiple layers connected together forms a structure to compute complex relationship that maps inputs to the target output. An overview is shown in **Figure 2.4**.

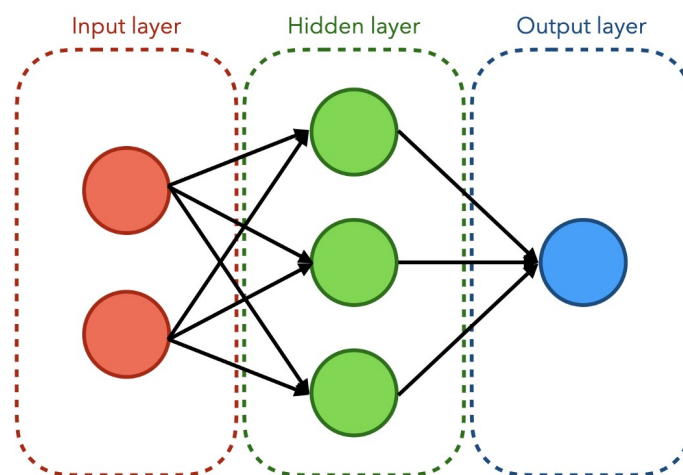


Figure 2.4: A Multilayer Perceptron Model

These type of simple architectures like the one shown in **Figure 2.4** are also called Multilayer Perceptron model or MLP. This specific MLP model consists of one input layer, one hidden layer and one output layer. The input layer shows that the input has two features. Each feature is then treated like a neuron activation to calculate the activation of the neurons or units of the hidden layer. Similarly, outputs (only one in this figure) are calculated with the activations of hidden layer neurons. The layers between the input layer and the output layer are called hidden layers because an ANN model takes inputs and returns the calculated outputs without showing the internal hidden outputs from a high level overview. The number of hidden layers can be very big depending on the problem in hand. These are called Deep Neural Networks or DNN. An illustration is given in **Figure 2.5**.

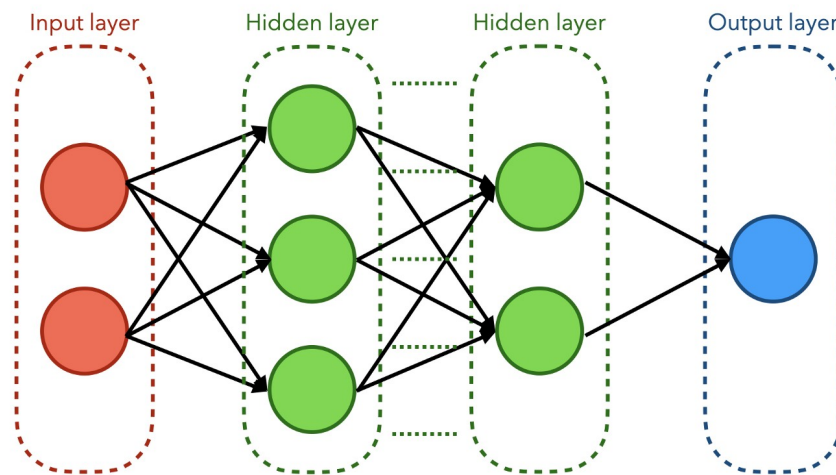


Figure 2.5: A Deep Neural Network

2.1.2 Forward Pass

Computations of the ANN layers are relatively simple. As stated earlier, each neuron in a layer computes logit ($\ddagger$) with a linear combination of incoming activations. The parameters for the combination are unique for each neuron and called weights. These weights and biases are learned via the training process. Calculating the activations of outputs from inputs is called the forward pass. Calculation of the forward pass are as follows:

$$Z_n = X_{n-1} \cdot W_n^T + b_n$$

$$X_n = f_n(Z_n)$$

Here,

X := Activations

Z := Logits

W := Layer weights

b := Layer bias

$f(.)$:= Activation function

The subscript denotes the current layer number; i.e., X_1 is the activations of the first hidden layer and X_0 denotes the actual input samples. Neural Networks are usually trained on batches of sample data instead of a single one. This enables faster training speed and convergence utilizing parallelism of modern hardware. The equations above uses batch of samples as inputs and also computes all outputs in parallel. This requires using multidimensional arrays or tensors. All of the parameters here are tensors.

To calculate the activations of the first hidden layer, we first calculate the logits, $Z_1 = X_0 \cdot W_1^T + b_1$. For example, if we take 16 samples in a batch and consider the architecture in **Figure 2.5**, then we have 16 input samples with 2 features making a batch shape of (16, 2) for X_0 . We will have a weight shape of (3, 2). These weights are variables and can be learned during training to better fit the data. Then, matrix multiplication with the transpose of W_1 and bias summation with $Z_1 = X_0 \cdot W_1^T + b_1$ will yield logits Z_1 of shape (16, 3). Activation function $f(.)$ works

element-wise on the tensor and does not change its shape. As logits are just linear combinations, an activation function is needed to introduce non-linearity. Otherwise, adding multiple layers would not achieve anything greater than using a single layer, as they will always be some linear combination of the inputs. Calculated activation X_1 's shape (16, 3) represents 3 activations of first hidden layer for the 16 input samples. This process is continued forward to get the output.

2.1.3 Loss Function

Initially, ANN model's weights and biases are initialized randomly [7][8][9] and the model generates random outputs. To achieve correct output, the model is then given a feedback with a function called the loss function. In a supervised learning scenario, where the true target output is given, the loss function can be seen as generating large values as penalty for very different outputs compared to the true target. The model tries to minimize the loss with a optimization method to train the model.

2.1.4 Optimization

The loss function is used to calculate the gradient with respect to the weights. These gradients are multiplied by a hyperparameter called learning rate and are used to update the weights in a backward pass through the model. This backward pass method is called backpropagation, which was popularized with Geoffrey Hinton's critically acclaimed paper in 1986 [10]. It made a surge on the interest of neural networks.

There are many different optimization algorithm involving backpropagation. Some of the most popular ones are Stochastic Gradient Descent or SGD [11], RMSprop [12] and Adam [13] optimizers. With a number of iterative updating or epochs, neural networks learn the optimal weights that fits the problem at hand.

To have a faster convergence the learning rate can be changed by a learning rate scheduler algorithm [14]. Batch normalization can also be used in deep neural networks to allow higher learning rates and faster convergence [15].

Sometimes, training data can become too simple for the model and the model overfits [16] the training data which can not generalize its training on unseen data. To remedy this, several regularization method are used. Most notables are the L2 Regularizer or Weight Decay [17][18]

and Dropout layers [19][20].

2.2 Recurrent Neural Network

2.2.1 Architecture

ANN architecture is the basis of every deep neural networks. The hidden layers shown in **Figure 2.5**, which transforms input activations in a linear space before applying activation functions are called Fully Connected Layers, more commonly known as Dense layers. But these are not the only type of hidden layers that can be used in DNN.

Dense layers works well only on data where each features are independent. More sophisticated layers are required for more sophisticated datasets. For example, in computer vision, input data are usually images. Each pixel of an image are treated as a feature. But as the pixels correlate with others, Convolutional Neural Networks or CNN [21] are used in hidden layers to catch these patterns.

When working with sequential data like time series data for stock price predictions, a rather complicated architecture is needed. Recurrent Neural Networks or RNN works on sequential data to catch sequential dependencies of the data. Language data is also sequential. Although, one dimensional CNN layers can catch some sequential patterns, NLP largely depends on RNN layers [22][23]. There has been many improvements on RNN and many new architecture has evolved on this idea over the years. We will briefly describe the simple RNN in this section.

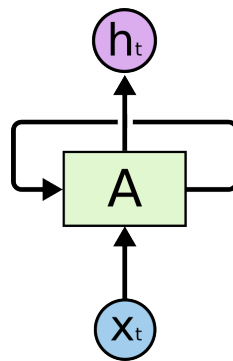


Figure 2.6: An RNN cell

Figure 2.6 shows a single unit or cell or RNN layer. They take sequence of inputs and produces a sequence of outputs. X_t is the input at timestamp t and h_t is the calculated hidden state

at timestamp t . Internally it looks like the illustration shown in **Figure 2.7**. Here, the subscripts denote the timestamps of the sequence.

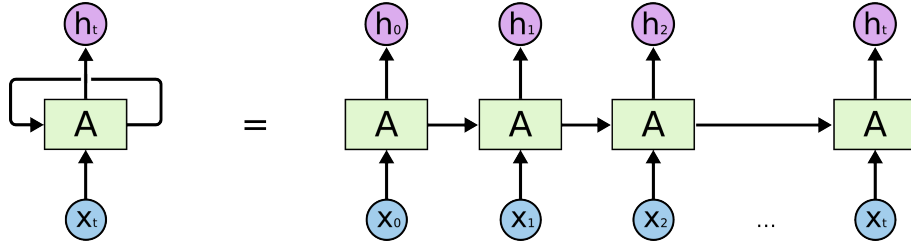


Figure 2.7: Unrolled view of an RNN cell

2.2.2 Forward Pass

The unrolled version of the cell is still a single cell. RNN cells need to learn extra weights for the hidden units. These hidden units keep the key information of a sequence or sentence; i.e., gender of a person, which is forwarded through the next words or timestamps to catch the underlying information of the whole sequence or sentence. So, an RNN cell takes two inputs: previous hidden state and sequence data, then produces next hidden state. Initial hidden state is generated randomly. In each timestamp, RNN concatenates the inputs and calculates the logits like a dense layer. Logits are passed through a tanh activation function [24] to calculate the new hidden state. This is illustrated from a high level overview in **Figure 2.8**.

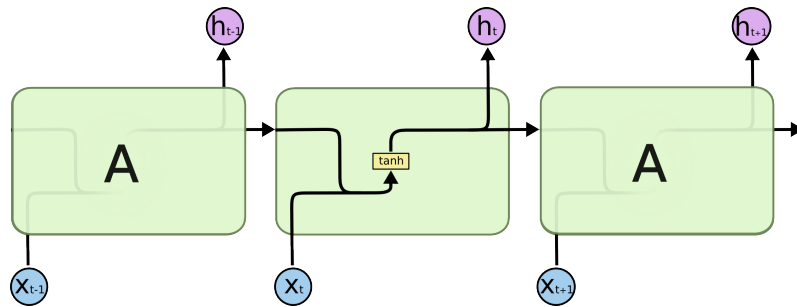


Figure 2.8: An RNN cell overview

RNN layers can also be stacked one after another like Dense layers. Which outputs are passed from one layer to another depends on the objective at hand. Sometimes just the last hidden states is passed and sometimes the collective hidden states. These different relations are shown in the **Figure 2.9**.

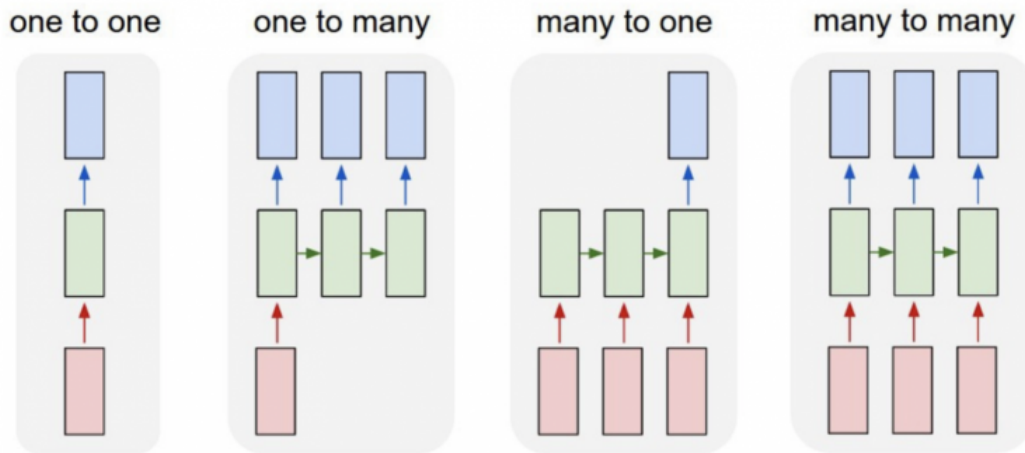


Figure 2.9: Recurrent Neural Network relations

For example, in sentiment analysis we have a Many-to-One relationship, where inputs are sequence of words and outputs are of single scalar value portraying the target sentiment. In machine translation; i.e., English to German translation, what we get is a Many-to-Many relationship. One-to-One relationship are the most basic and traditional form where one output is generated for a single input. One-to-Many relationship is generating a sequence of outputs for a single input. Music generation given a single starting note or a genre is a One-to-Many relationship.

Our objective on sentimental style transfer generates a sentence given a input sentence. It has a Many-to-Many relationship architecture.

2.2.3 Training

RNN are trained in a similar fashion like ANN with gradients. A loss function is used depending on the objective. Weights get updated while trying to minimize the loss. The gradients computation are more complicated here and computationally expensive. Also, RNN suffers from the lack of parallelization because of sequential data. This is why RNN training takes much longer time compared to ANN.

2.2.4 Drawbacks

RNN models keeps the sequential flow of information in hidden states. These hidden states get bottlenecked with long sequences. While updating the weights with gradient descent [11], as the gradients are also dependent on the sequence, gradient gets too large or too small. These are called exploding gradient and vanishing gradient problem respectively. These problems does not allow the model to train properly. RNN architectures have evolved with improved architectures to mitigate those problems. We will briefly demonstrate a few in upcoming sections.

2.3 Long Short Term Memory

2.3.1 Architecture

Long Short Term Memory, more commonly known as LSTM [22], modifies the RNN architecture and improves it to tackle exploding or vanishing gradient problem [25]. LSTM keeps an extra state besides the hidden state called the cell state to better contain the flow of information. Architecture of an LSTM cell is shown in **Figure 2.10** below.

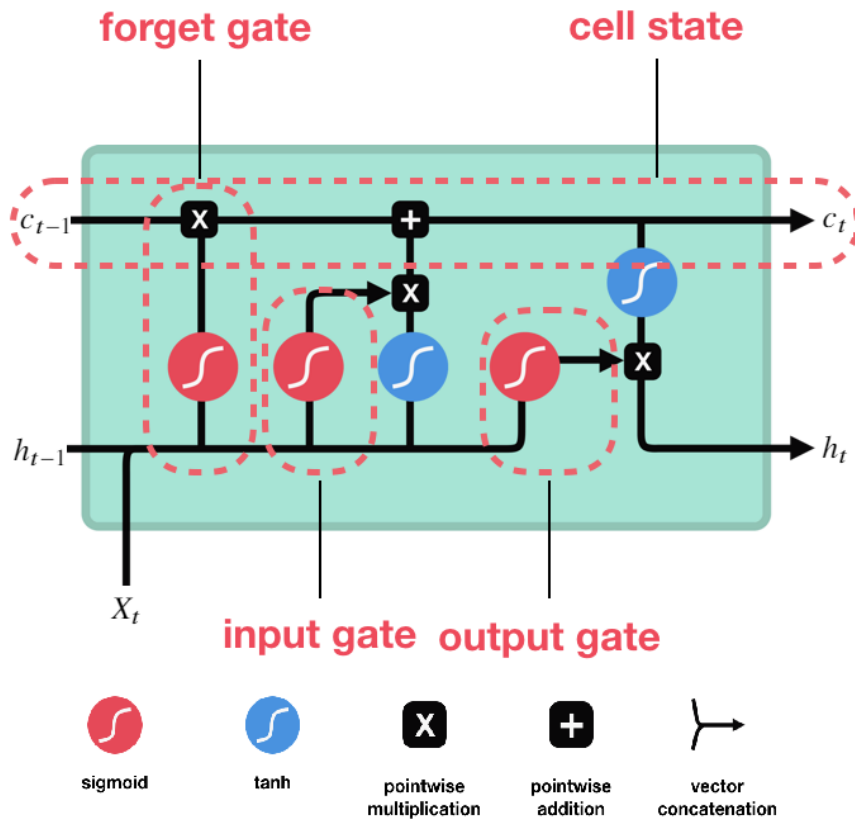


Figure 2.10: An LSTM Cell

Behind the scene, this complex architecture of a LSTM cell is composed of multiple Dense layer computations and interactions between them. The cell state of LSTM acts like the memory of the LSTM cell. c_t in the figure above denotes cell state at timestamp t . They contains relevant information where information gets added or removed through the sequence via the gates. These gates are computing units of LSTM. All of them uses sigmoid activation functions to condense the values between 0 and 1. Forget gates removes the irrelevant information from the previous cell

state. Input gate transforms the concatenated values of previous hidden states and current inputs through a dense computation with sigmoid activation function. It gets multiplied elementwise with a candidate cell state having tanh activations. This gets added to the cell state that has been previously updated via the forget gate. This output is the new cell state of the current LSTM cell. To calculate the hidden state, this new cell state and output gate is used.

The illustration **Figure 2.10** is of a single timestamp of LSTM cell. An unrolled view of the cell is illustrated in **Figure 2.11** similar to a RNN cell.

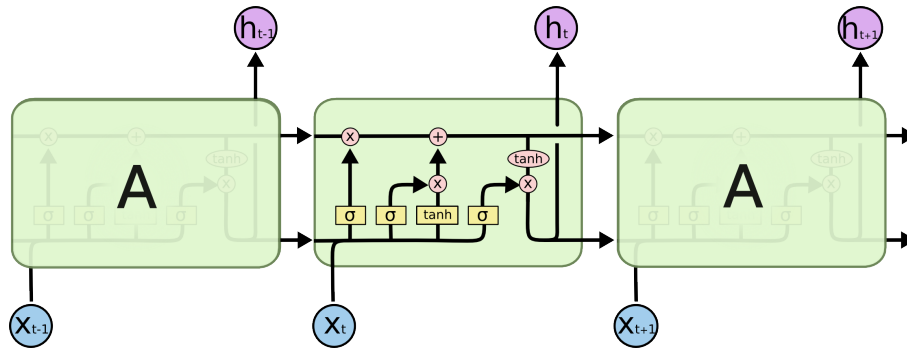


Figure 2.11: Unrolled view of an LSTM Cell

2.3.2 Improvements over Vanilla RNN

The cell state introduced in LSTM can flow through a long sequence without much modifications. In **Figure 2.11**, it can be seen from the horizontal line that connects the cell states of different timestamps. During training gradient can flow very easily through the cell states, as it does not involve any non-linear activations in its way. This helps in mitigating the vanishing gradient problem of vanilla RNN, which in turn helps LSTM catching long term dependencies in input sequences.

2.3.3 Drawbacks

Although LSTM gives a fine structure for sequential data, it still has its limitations. On very long sequences like articles or books, LSTM suffers because of bottlenecking in the hidden state. The hidden states are sequential and cannot utilize modern computation powers by parallelization. This makes LSTM very slow to train.

In 2017, a new model architecture called Transformer is introduced by a team from Google in the very successful and popular paper “Attention is all you need” [26]. The transformer model uses attention for sequence data and tackles the two problems of LSTM mentioned above. This new architecture revolutionized the field of Natural Language Processing in recent years and achieves state of the art results in almost every domain of NLP.

2.4 Gated Recurrent Unit

Gated Recurrent Units or GRU are introduced much later than LSTM, in 2014 [27]. GRU tries to capture the model architecture of LSTM in a simpler way. It does not use two different states like LSTM and uses two gates instead of three. An overview of GRU cell is shown in **Figure 2.12**.

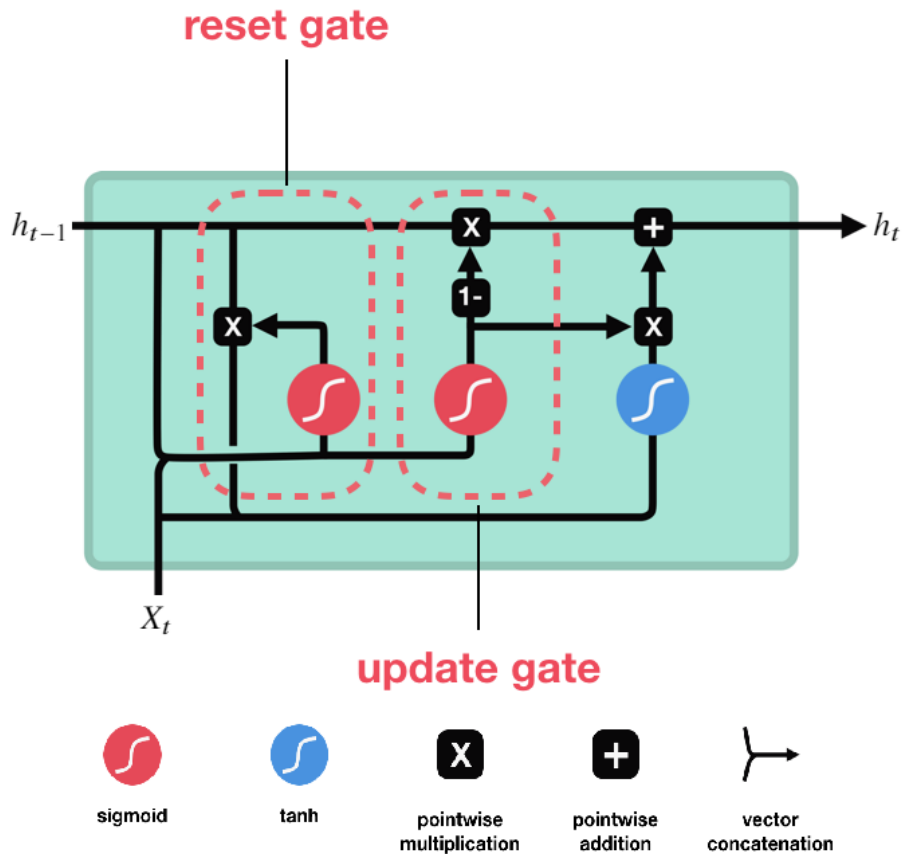


Figure 2.12: Overview of a GRU Cell

GRU uses a single activation and its complement to perform similarly like a LSTM cell state's forgetting and adding functionalities. It is faster to train and often achieves same level performance of LSTM.

2.5 Autoencoder

2.5.1 Architecture

Autoencoders [28] are used to learn data representation in an unsupervised manner. They usually have their inputs as the targets and try to reconstruct the given input while passing through a bottleneck layer. Autoencoders consist of two different models for encoding and decoding, one being the opposite of another. They are mostly used in computer vision for compressing images in a latent space. An overview is given in **Figure 2.13**.

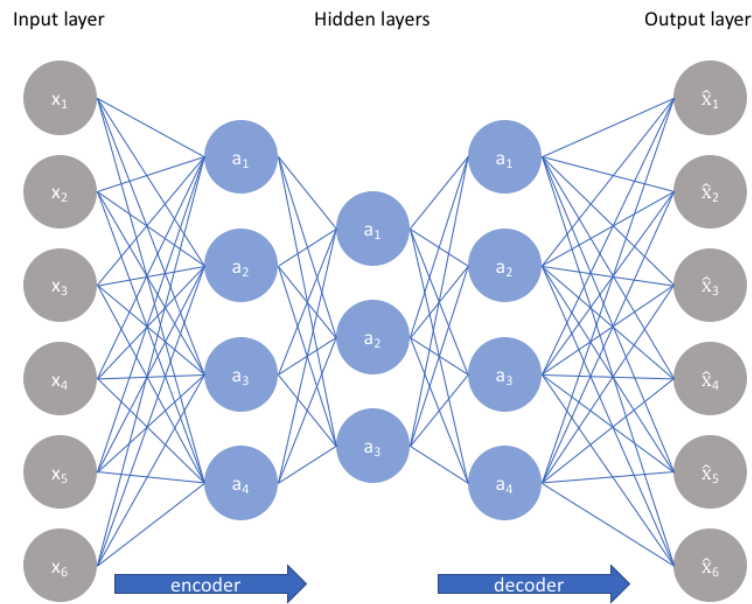


Figure 2.13: An Autoencoder with 3 dimensional latent space

This Autoencoder has 6 input features. These features are compressed to 3 in the middle layer, which is called the latent space. Through the computations and transformation of previous layers input information are bottlenecked in the latent space. This first part of the Autoencoder is called the Encoder. Data from the latent space is used in a reversed manner to reconstruct the input image. This second part of the model is the Decoder of an Autoencoder.

2.5.2 Training

The encoder network is usually denoted by a function $g(\cdot)$ parameterized by ϕ . The decoder network is denoted by a function $f(\cdot)$ parameterized by θ . The encoder generates latent vector z

from input x ,

$$\mathbf{z} = g_{\phi}(\mathbf{x})$$

The decoder generates reconstructed sample x' from z ,

$$\mathbf{x}' = f_{\theta}(\mathbf{z})$$

This can be generalized as,

$$\mathbf{x}' = f_{\theta}(g_{\phi}(\mathbf{x}))$$

As the target of Autoencoder is to minimize the difference between input data and reconstructed data, simple mean squared error works quite often. But this can be different depending on the context scenario. This loss is also called the reconstruction loss an Autoencoder. Autoencoder's loss, $L_{AE}(\theta, \phi)$ can be calculated as follows in case of Mean Squared Error or MSE,

$$L_{AE}(\theta, \phi) = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}^{(i)} - f_{\theta}(g_{\phi}(\mathbf{x}^{(i)})))^2$$

Here, n is the number of features and $\mathbf{x}^{(i)}$ denotes the i^{th} feature.

2.5.3 Use cases

Although, this might seem useless to generate the same given input, Autoencoder's usefulness lies in the bottlenecked layer or latent space. The latent data can be extracted as a compressed version of the input [29]. These compressed data can later be used to reconstruct the original input. The reconstructed input is not lossless, but unlike the illustration shown in **Figure 2.13**, which only reduces the input dimension by half, Autoencoders can compress images or other data in a much greater ratio.

Autoencoders can also be used to retrieve lost information from data like denoising images [30]. For this purpose, noisy images are given as encoder inputs, but the reconstructed images are compared to the actual images to calculate the loss.

2.6 Variational Autoencoders

2.6.1 Probabilistic Models

Most of the machine learning models work in a discriminative fashion. For some given input data, a fixed output is generated. These type of models are called discriminative models. In supervised learning, this type of model works very well and a generative model is often not necessary. But discriminative models are not probabilistic, hence can not learn the uncertainties related to the inputs or outputs. For example, a deterministic linear regression model could fit a line to the sample training data, but given a different sample of the data, the model could fit a different line. This type of uncertainty in the data is called epistemic uncertainty. This uncertainty decreases as the training dataset increases in size.

Another type of uncertainty is called aleatoric uncertainty [31][32]. For the same linear regression model mentioned above a fitted line always gives a single value for a particular input in a discriminative model. These model fails to show the possible variance of the output.

A probabilistic model tries to show these uncertainties of the model. In a sense, a probabilistic model knows what it does not know. Probabilistic model also generates sample distribution of the output rather than a fixed output. There are certain probabilistic models which can generate new samples after learning the probability distribution of the datasets. These models are called generative models. Variational Autoencoder is one of them.

2.6.2 Architecture

From a high level overview, a Variational Autoencoder or VAE looks almost same as an Autoencoder. It works in a similar fashion too. Given the training data, a VAE model tries to reconstruct the data. But unlike Autoencoders, VAE generates probabilistic distribution for the latent space. The encoder in VAE is probabilistic and by sampling from distribution, decoder can generate similar but new samples of the data. **Figure 2.14** illustrates how a Variational Autoencoder works.

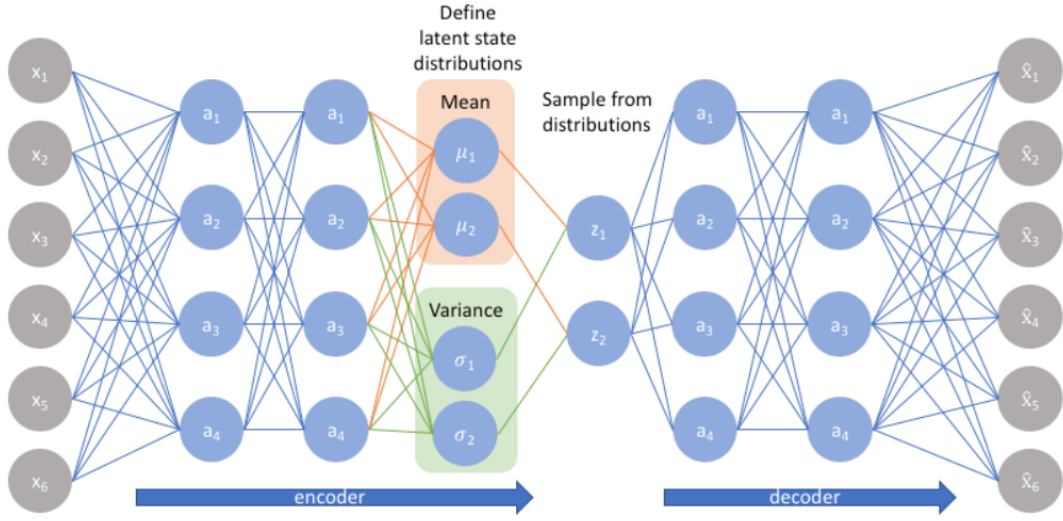


Figure 2.14: A Variational Autoencoder with 2 dimensional latent space

Similar to Autoencoders, the encoder is used to generate latent space. But instead of fixed values, VAE encoder finds mean, μ and variance, σ . This generates latent space $q_{\phi}(\mathbf{z}|\mathbf{x})$, where ϕ is the learned parameters of the encoder. Sample, $\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})$ is given as the input of the decoder to generate new sample, $x' \sim p_{\theta}(\mathbf{x}|\mathbf{z})$, θ being the learnable weights of the decoder. These samples are similar to the actual input data.

Training a VAE requires more refined algorithms, as it introduces many new complexities with probabilistic features. These methods are elaborated while describing the methodology of our model in *Chapter 4*.

2.7 Generative Adversarial Network

Generative adversarial networks or GANs are a powerful type of generative model used for unsupervised machine learning. They were first introduced by Ian Goodfellow in 2014 [3]. Goodfellow developed them to tackle some of the issues with similar neural networks, including the Boltzmann machine [33] and Autoencoders [34]. Both the Boltzmann machine and Generative Autoencoders use the markov decision chain [35], which has a pretty high computational cost. GANs provide efficiency over them by providing significant gains when working with the cutting edge of artificial intelligence. Since then, GANs have been used in numerous ways, giving birth

to many state of the art results, specially in Computer Vision Domain. GANs main success was showcased by generating fake data. It has achieved unparalleled performance on fake image generation; i.e. generating fake human faces or animal images. **Figure 2.15**, shows some fake celebrity images generated by StyleGAN2 [36] [37]. Random samples can be obtained from the referred website [38].



Figure 2.15: Fake celebrity images generated by GAN

2.7.1 Architecture

A Generative Adversarial Network consists of two separate neural networks: **Generator** and **Discriminator**. Unlike usual neural networks, these two counterparts of GAN acts in a competitive way trying to beat each other. Generator tries to generate fake samples of data, while discriminator acts as a classifier to detect if the given sample is fake or real. A GAN architecture for fake image generation is shown in **Figure 2.16**.

GANs works on their objective in a very intuitive manner. Let's say the GAN shown in **Figure 2.16** wants to generate fake paintings of Leonardo da Vinci. Training starts with the discriminator. It is trained as a classifier to classify real paintings and random noises. The generator begins training by taking random noises from a distribution; i.e, an uniform distribution. As expected, it generates some random noises as outputs which are nothing like a painting. The discriminator takes

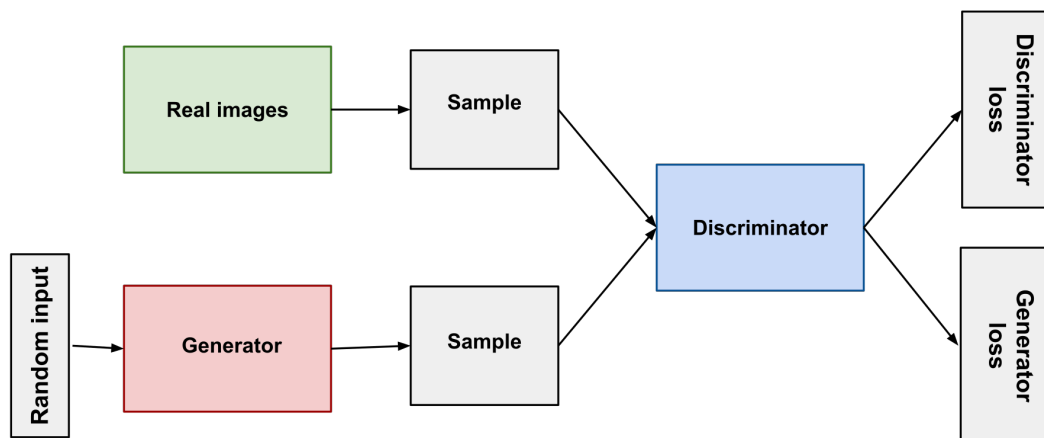


Figure 2.16: Generative Adversial Network

some real paintings along with the fake noises from generator. It easily classifies them correctly. The generator gets feedback through the discriminator during backpropagation and updates its weight to make a better attempt on generating fake paintings. After enough updates, the generator can generate good enough outputs that can fool the discriminator.

2.7.2 Training

Although it sounds simple enough to train, there are certain caveats when training a GAN. Both of the generator and discriminator are usually trained on the same loss function. While one tries to minimize it and the other tries to maximize. For example, the generator can try to minimize the difference between the distribution of generated sample and the distribution of real samples. While, discriminator tries to maximize the difference.

GAN trains in a wake-sleep manner. During discriminator training, generator's weights are kept frozen. As discriminator training tries to figure out how to distinguish real data from fake, it has to learn how to recognize the generator's flaws. That is a different problem for a thoroughly trained generator than it is for an untrained generator that produces random output.

Similarly, discriminator's weight are not updated during generator's training. This way, gradients from discriminator do not move continuously in different direction, which can hurt generator's training process.

As the generator gets better with each iteration, the discriminator performs worse, as it can gets

harder to spot the difference between real and fake. This poses a problem for the GAN architecture as a whole. With increasingly difficult samples from generator, discriminator can not give proper feedback. As, the generator's learning is completely based on discriminator's feedback, it can degrade in performance in certain scenarios.

2.7.3 Variations and use cases

There has been numerous research on GAN since its arrival in 2014 and researchers continue to find different possibilities of it. We showcase a few of them here.

Instead of learning the joint distribution of the data, GAN can also be trained to generate samples based on condition. For example, instead of generating random celebrity images, GAN can be conditioned on generating images with specific attributes; i.e. gender, ethnicity, hair color etc. This type of GANs are known as Conditional GANs [39].

CycleGAN can transform inputs from one distribution to a similar one without any parallel training data or supervision [40]. For example, transforming an image of a horse to a zebra. This is shown in **Figure 2.17**.



Figure 2.17: CycleGAN transforming a horse to a zebra

GANs can also be used for image super resolution [41], image-to-image translation [42], face inpainting etc [43]. Besides these amazing use cases in Computer Vision, GANs can also be deployed for tasks like understanding risk and recovery in healthcare and pharmacology. GANs are also being used in fields such as drug research. They can be trained on the existing drugs and suggest new synthetic chemical structures that improve on drugs that already exist.

GANs has gained comparatively less success in the field of NLP. They can be used for text-to-image synthesis [44] and text-to-speech [45] etc. With various use cases GAN's training algorithm and loss function also varies drastically. For our objective of textual style transfer, we have decided to use a hybrid structure combining VAE and GAN. Optimization procedure for this becomes quite complex, which is elaborated while describing the methodology of our model in *Chapter 4*.

Chapter 3

Literature Review

We studied several papers and articles with remarkable results regarding style transfer of text, based on both parallel and non-parallel data. Among them the following papers are the most related ones to our work. We learned fundamental theories about the subject of our interest and also looked at the implementation of these theories to achieve our goal.

3.1 Relevant Works relating to our objective

3.1.1 Sequence to Better Sequence: Continuous Revision of Combinatorial Structures

This paper [46] focuses on producing revised output sentence based on the input sequence, keeping similarities about the concept but also improving the positivity of the input sequence. This paper proposes a generative model with Variational Auto-Encoder which helps to learn continuous latent factors and also a neural network based module for the outcome prediction. The VAE decoder generates the revised sequence based on optimizations and maintains similarities with the input. Here, all the RNNs employ GRU to learn more efficiently about the long-range dependencies of the input sequence. Gradient based methods are implemented to optimize the continuous latent factors.

3.1.2 Decomposing Textual Information For Style Transfer

This paper [47] works with two extensions to the work of the paper [48]. The original structure of the model consists Auto-Encoder with one encoder and one generator. The input sentence is encoded into a latent representation by the encoder and the latent representation along with the target style are passed into the generator. Then the generator generates the output sequence and also keeps track if the output matches the target style by a discriminator.

In this paper, they added two things to the baseline model structure. First one is adding another dedicated discriminator that will make sure that the latent space representation contains no information about the style. And the second one is calculating a loss component, while processing the output sequence by the encoder to verify the semantic similarities between the input and output sentence. This second structure is named Shifted Auto-Encoder or SAE. At first, they experimented the model using the first extension or adding another discriminator only. Secondly, they used SAE approach too and finally combined both.

3.1.3 Delete, Retrieve, Generate: A Simple Approach to Sentiment and Style Transfer

Here [49] the method is based identifying attribute markers from unaligned corpus. In this paper overall two baselines ‘RETRIVEONLY’ and ‘TEMPLATEBASED’ and two neural models ‘DELETEONLY’ and ‘DELETEANDRETRIVE’ are proposed. From the inputs to delete n-grams that have the most discriminating power (which indicates the sentence is positive or negative) a delete function is proposed. To fill up the deleted n-gram portion and reproduce the sentence a retrieve method is defined where they used two methods for word mapping. One is TF-IDF weighted word overlap and the other is Euclidean distance. So, it decides what words or phrases to insert in the sentence to reproduce the sentence with same context but with target attribute (positive or negative).

3.1.4 Sentiment Transfer using Seq2Seq Adversarial Autoencoders

This paper [50] proposes an Adversarial Sequence to Sequence model that is trained on non-parallel corpus. The structure of this model includes Autoencoder and GAN also. This method is quite similar to the method used in the paper [46]. The only difference is that in that paper

VAE was used instead of Auto-Encoders. Here, the inputs are turned into word vectors then they are encoded into a latent space and decoded using a generator with discriminator to get the target output. They also experimented with other sequence to sequence models such as Vanilla, Reduced, Reversed, Bidirectional, Reduced Bidirectional etc.

3.1.5 Unpaired Sentiment-to-Sentiment Translation: A Cycled Reinforcement Learning Approach

To achieve the goal of sentimental style transfer in text this paper [51] introduces a cycled reinforcement learning method. This method combines two modules called neutralization and emotionalization. The collaboration between these two modules enables the model to be trained on unpaired data. They evaluated their approach with two datasets, yelp and amazon. And also compared their results with other existing models such as CAAE and MDAL and overall their methods outperforms these models.

3.1.6 Learning Sentiment Memories for Sentiment Modification without Parallel Data

Here [52] the main model structure is based on Sentimental Memory based Auto-Encoders. In the paper [49] a function was proposed for identifying positive or negative attribute (n-grams) markers. Similarly, the model structure of this paper includes sentiment based two memories positive memory and negative memory. For any input sentence its embedding is passed into some attention mechanism to find the columns of memory (positive or negative). Then the outer product of the embedding of the sentence and attention weights are calculated and memory is updated with that. Then finally the generator generates the output sequence and the sentimental memories assist them in generating.

3.1.7 Contextual Text Style Transfer

Their work [53] mainly focuses on contextual style transfer to a sentence. For this purpose they introduced Context-Aware-Style-Transfer (CAST) model which uses two different encoders. One for the input sentence and the other for the desired context. They worked with both parallel and non-parallel corpus. So, they used two types of CAST model, one for parallel data (supervised

approach) and the other for non-parallel data (unsupervised approach). The structure of the supervised approach includes a context encoder and a sentence encoder that encodes given context and input sentence respectively. Then the output of these encoders are passed onto the sentence decoder along with the target style. The output is tested with a classifier to determine whether it maintains the contextual consistency with the input context.

On the other hand, in the unsupervised approach, sentences from the non-parallel corpus are taken as inputs to a sentence encoder. Then the encoded output is passed as input to two separate sentence decoders along with original style and target style respectively. After that output of these decoders are compared by a classifier and reconstruction loss and style classification loss are calculated. The output of the sentence decoder with target style is again encoded and then re-decoded with the original style to get the back transition loss. Hence, optimizing these loss functions target sequence is generated.

3.1.8 Domain Adaptive Text Style Transfer

Two Domain Adaptive Style Transfer (DAST) models are proposed here [54]. Both of them does two style transfer tasks such as sentiment transferring and formality transferring. The two models DAST and DAST with content preservation (DAST-C) works on non-parallel data. The structure of both the models includes two parallel shared Encoder and shared Decoder to shift the input from source domain to target domain. The only difference is DAST-C learns the generic content through reconstruction objective parameter defined on massive domain data with unknown style. The models are compared with previous approaches over multiple domain targets also.

3.1.9 IMaT: Unsupervised Text Attribute Transfer via Iterative Matching and Translation

The approach of this paper [55] represents a simpler model called Iterative Matching and Translation (IMaT). Since there is a lack of supervised parallel data which makes text style transfer a bit complicated, this paper introduces a pseudo-parallel corpus by pairing a subset of similar sentences from corpora that has different attributes. Here, the attributes can be positive, negative, formal, informal etc. They use cosine similarity to compare the semantic similarities between two sets of sentences from different attribute corpora and align them. Hence, they are able to create

a semi-parallel data. And using that they created a semi-supervised model based on sequence to sequence model. Their model structure contains two layers of LSTM encoder-decoder with 1024 hidden dimensions and attention. Their approach outperforms existing methods with 95% accuracy.

3.1.10 Sentimental Style Transfer in Text with Multigenerative Variational Auto-Encoder

The model structure described in this paper [5] uses VAE which doesn't only increase the positivity of the input sentence like the paper [46] they generate both negative and positive sentences. They also experimented with vanilla Auto-Encoders but it didn't work as well as multigenerative auto-encoders. That's why the main model structure uses VAE with two generators. In general, VAE structure has one encoder and one decoder. Encoder encodes the sentence and the decoder generates new sentence based on the latent space inputs. But in this model they modified VAE structure with two decoders, one for positive sentence generation and another for negative sentence generation. They worked with Bangla Dataset collected from several newspapers.

In our work we experimented with extensions of this model. We intend to experiment with different dataset to improve the accuracy and modify the VAE structure with a discriminator.

3.2 Other notable researches

Almost all of the mentioned papers in **Section: 3.1** works with non-parallel data. Among the mentioned papers in **Section: 3.1**, the paper "Contextual Text Style Transfer" [53] works with both parallel and non-parallel corpus. The paper mentioned in **Section: 3.9** [55], works with pseudo-parallel corpus. Our work was also based on non-parallel data and the mentioned papers in **Section: 3.1** match our purpose closely. We have also studied some other notable papers that worked on parallel data [56] [57] [58]. Besides these papers, there are many other researches, that have worked on this domain. Some of the interesting works are Natural Language Generation [59], Sequence to Sequence models etc.

Chapter 4

Methodology

Style Transfer of Text is a Sequence-to-Sequence machine learning problem. Unlike other Seq2Seq models [60]; i.e., machine translation models [61], style transfer needs more refined approach to be generative. We have explored quite a few of those on our way to select our final model. It also required exploring different subtasks like word embeddings [62] to best fit our model. In this chapter, we will first share these exploration first, that helped choosing our model. Then, we will try to give a broad overview of our final model.

4.1 Exploring Candidate Models

4.1.1 Word Embeddings

Machine Learning models require numbers to work with. They do not recognize colors of an image or words of a sentence. These details need to be somehow encoded with numbers to help the model utilize their meanings and distinguish those to build a structure based on different machine learning objectives. Image colors for a pixel are usually stored as 3 separate values for red, green and blue color intensity. A model can utilize this values for its purpose. Language is not that simple. To represent words in a number space more sophisticated methods are developed. These are called word embeddings.

Word embedding tries to represent a single word in multidimensional vector space. At the time of writing this, the most acclaimed model for NLP is Generative Pre-trained Transformer 3 or GPT-3 [63]. It uses an embedding vector of length 2048 to represent a single word. These values

of the embedding is learned during training. Two of the most common methods are:

- Continuous Bag of Words or CBOW.
- Skip-gram

These methods essentially work in a similar fashion. An illustration of CBOW is given in

Figure 4.1.

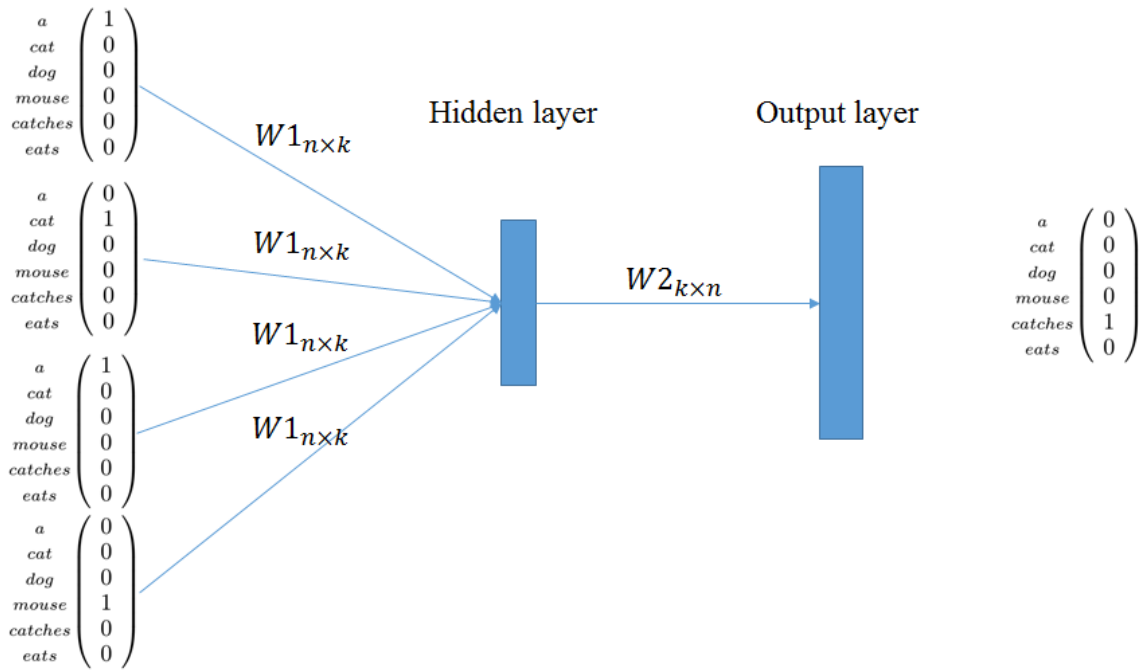


Figure 4.1: Continuous Bag of Words with 4 Context Words

Figure 4.1 above shows how a CBOW embedding is trained with 4 context words. Given the surrounding context words the model tries to predict the target word. Here, given “A cat ____ a mouse” as inputs, the embedding is learned to predict the word “catches”. The input words are given as one-hot-encoding of the words [64]. The weights in the middle that transforms the given input to output is the learned embedding.

Skipgram works in similar fashion but in a reversed manner. It tries to predict the surrounding context, given a word. An illustration is given below in **Figure 4.2**.

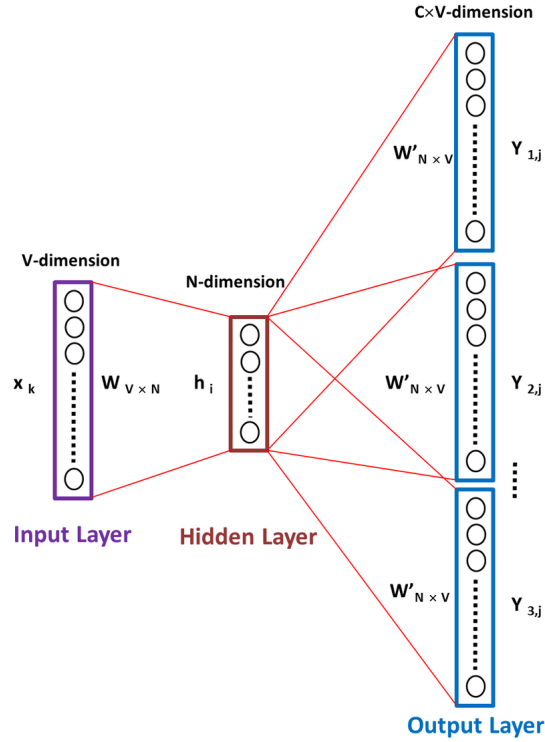


Figure 4.2: Skipgram Embedding

These type of embeddings contain inherent meaning of words by containing specific information on specific direction of the vector space. For example, in a good word embedding trained with enough data, we can get a relation like below,

$$\begin{aligned} \text{King} - \text{Queen} &= \text{Man} - \text{Woman} \\ \implies \text{King} - (\text{Man} - \text{Woman}) &= \text{Queen} \end{aligned}$$

The vector obtained from $(\text{Man} - \text{Woman})$, can be used to obtain opposite gender of a word like above.

Another popular embedding is GloVe vectors [65]. It is trained with Linear Discriminant Analysis [66] and tries to catch global co-occurrence statistics of words from a given corpus.

Deep learning frameworks like TensorFlow or PyTorch has dedicated embedding layers, that can be learned during training on a given corpus. We tried this approach first, but it did not work

well for our small text corpus. We later experimented with multiple pretrained embeddings, which gave us a much better result.

4.1.2 Seq2Seq

Typical Seq2Seq models work very well with supervised training data. They have two separate parts: Encoder and Decoder. Both of those use RNN layers. Encoder encodes the input sentences and the Decoder takes the hidden states outputted from the Encoder as inputs to produce the target sentence. An overview of these type of model is shown in **Figure 4.2**.

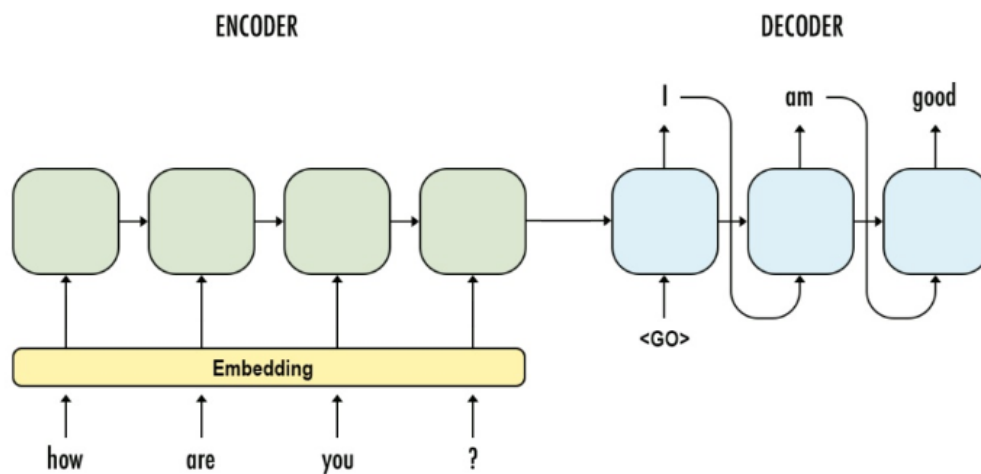


Figure 4.3: Overview of a Sequence to Sequence Model

We experimented with these naive Seq2Seq models with multi-decoder architecture for different sentiments using different RNN layers like LSTM or GRU. They did not give us a desired result and we decided to move on to more sophisticated model architecture with generative models.

4.1.3 Generative Models

Generative models are much more powerful than our earlier experimented models like Seq2Seq. We have finally started exploring these and found that these models fit our objectives very well. First, we used Variational Autoencoders with attribute vectors to discriminate between positive and negative sentiment sentences. It showed significant improvements on text generation. But

transferring sentiment with attribute vectors did not work very well and the model struggled with changing polarity. To focus on the polarity better, we decided to introduce a discriminator to the existing variational autoencoder. This gave us the final model for our purpose, a hybrid model of VAE and GAN.

4.2 Proposed Model

As our proposed model consists of both VAE and GAN, we will try explaining the underlying techniques for optimizing them first, before discussing the hybrid one.

4.2.1 Difference between Autoencoder and VAE

Variational Autoencoder or VAE is a probabilistic implementation of Autoencoder. It produces a probabilistic distribution of latent space instead of a distinct output. The distribution enables sampling from the latent space for the decoder, which in turn can generate new data that the model did not see during its training. This difference on latent space is shown in **Figure 4.4**.

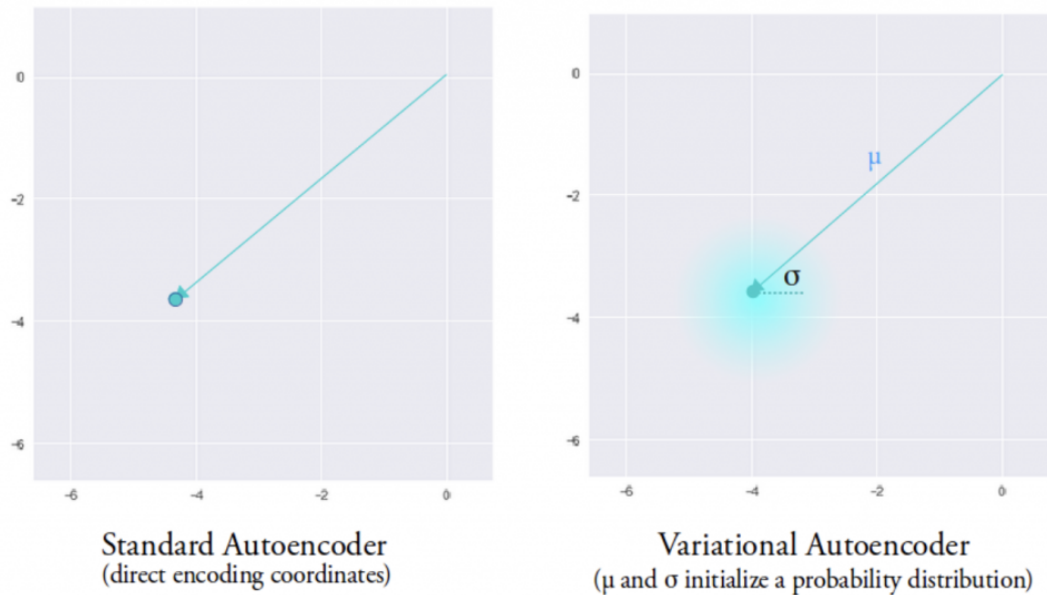


Figure 4.4: Difference of latent space between Autoencoder and VAE

Training an Autoencoder is similar to normal deep learning models. With probabilistic models

like Variational Autoencoders, new difficulties arise relating to gradient flow. These are described next.

4.2.2 Difficulties in Training Variational Autoencoders

Variational Autoencoder's idea is deeply rooted in Bayesian methods. Bayesian methods can be used to calculate the distribution of a model parameter given some data. This is convenient while working with weight uncertainty in neural networks, since we are looking for the distribution of weights or model parameters given some training data. The key step relies on Bayes theorem [67]. This theorem states that

$$P(Z|X) = \frac{P(X|Z)P(Z)}{P(X)} \quad (4.1)$$

Here:

- X is the input data. $P(X)$ is called the **evidence**.
- Z is the latent space.
- $P(Z)$ is called the **prior**. This is our "prior" belief on the probability density of a model weight. This is usually a simple known distribution like unit normal and is not learned.
- $P(X|Z)$ is called the **likelihood**. It shows the likelihood of having observed data X given latent values Z .
- $P(Z|X)$ is the **posterior**. It is called posterior since it represents the distribution of the latent space after taking the training data into account.

While **Equation 4.1** works in principle, the normalizing term or the **evidence** is often untractable or computationally very expensive because it involves integral of marginal distribution of the data over model's weights. This is shown in **Equation 4.2**

$$P(X) = \int P(X|z)P(z)dz \quad (4.2)$$

For this reason, an approximate method or Variational Bayes is used.

4.2.3 Variational Bayes

Variational Bayes [68] approximates the actual posterior with a different distribution called the **variational posterior**. It is selected in a way, such that it has known distribution, often a gaussian. The variational posterior has a number of parameters, denoted by ϕ , that are tuned so that it approximates the actual posterior as well as possible. Instead of finding actual posterior, $p_{\theta}(z|x)$, the model tries to find $q_{\phi}(z|x)$ as the variational posterior.

Overview of the architecture is shown in **Figure 4.5**. Different parameters of the model showed in the figure will be explained in the upcoming sections.

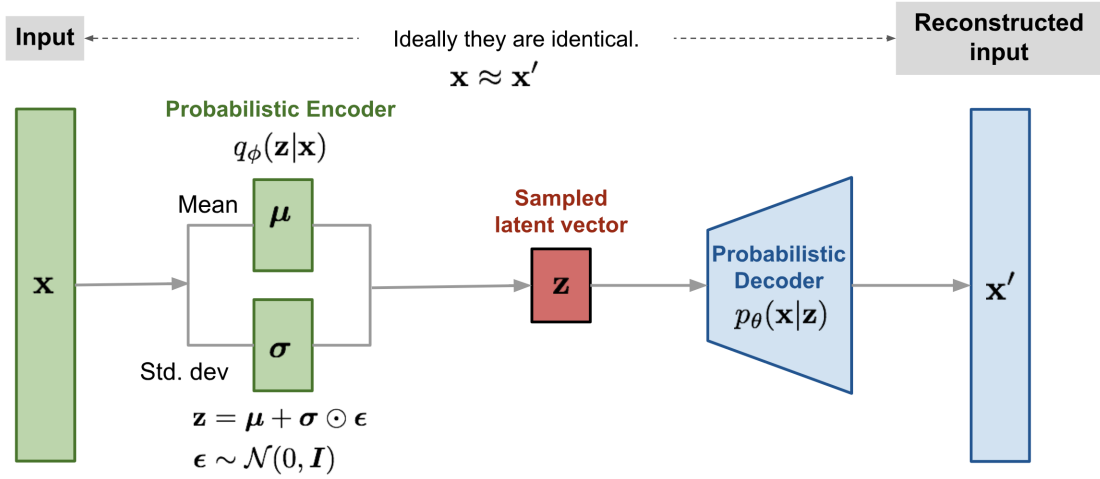


Figure 4.5: Illustration of the VAE model

4.2.4 Kullback Leibler Divergence

Kullback Leibler Divergence or KL Divergence [69] is a method to measure the difference between two distribution. KL Divergence between two distribution $f(x)$ and $g(x)$ is given by,

$$D_{KL}(f(x)||g(x)) = \int f(x) \log \frac{f(x)}{g(x)} dx. \quad (4.3)$$

KL divergence has some important properties:

- KL divergence is non-negative or, $D_{KL}[q || p] \geq 0$. It is zero only when the distributions are same almost everywhere. This is known as Gibb's inequality [70].
- KL divergence is not symmetric or $D_{KL}[q || p] \neq D_{KL}[p || q]$.
- KL divergence, $D_{KL}[q || p]$ is penalized heavily when support of q covers more space than support of p . So, $\text{support}(p) \subset \text{support}(q) \implies D_{KL}[q || p] \sim \infty$.

We can use KL divergence of the actual posterior and variational posterior and try to keep it close to zero to approximate a good posterior. From **Equation 4.3**, we have the KL divergence of the posterior and variational posterior as,

$$D_{KL}(q_\phi(z|x)||p_\theta(z|x)) = \int q_\phi(z|x) \log \left(\frac{q_\phi(z|x)}{p_\theta(z|x)} \right) dz \quad (4.4)$$

$$= \int q_\phi(z|x) \log \left(\frac{q_\phi(z|x)p_\theta(x)}{p_\theta(x|z)p_\theta(z)} \right) dz ; \text{ Bayes Theorem} \quad (4.5)$$

$$= \int q_\phi(z|x) \log p_\theta(x) dz + \int q_\phi(z|x) \log \left(\frac{q_\phi(z|x)}{p_\theta(z)} \right) dz - \int q_\phi(z|x) \log p_\theta(x|z) dz \quad (4.6)$$

$$= \int q_\phi(z|x) \log p_\theta(x) dz + D_{KL}(q_\phi(z|x)||p_\theta(z)) - \mathbb{E}_{z \sim q_\phi(z|x)}(\log p_\theta(x|z)) \quad (4.7)$$

Here, $\mathbb{E}_{z \sim q_\phi(z|x)}(\log p_\theta(x|z))$ is the expected value of the log likelihood over distribution

$q_\phi(z|x)$. As, the evidence, $p_\theta(x)$ is constant for a fixed data, we get:

$$\begin{aligned} D_{KL}(q_\phi(z|x)||p_\theta(z|x)) &= \log p_\theta(x) + D_{KL}(q_\phi(z|x)||p_\theta(z)) \\ &\quad - \mathbb{E}_{z \sim q_\phi(z|x)}(\log p_\theta(x|z)) \end{aligned} \quad (4.8)$$

We can ignore the constant term while minimising the function in **Equation 4.8**. So, we have the loss function of VAE as:

$$\mathcal{L}_{VAE} = D_{KL}(q_\phi(z|x)||p_\theta(z)) - \mathbb{E}_{z \sim q_\phi(z|x)}(\log p_\theta(x|z)) \quad (4.9)$$

4.2.5 Evidence Lower Bound

As stated earlier KL divergence is non-negative. Following this, from **Equation 4.8** we get something interesting about the log evidence,

$$\begin{aligned} \log p_\theta(x) + D_{KL}(q_\phi(z|x)||p_\theta(z)) - \mathbb{E}_{z \sim q_\phi(z|x)}(\log p_\theta(x|z)) &= D_{KL}(q_\phi(z|x)||p_\theta(z|x)) \\ \implies \log p_\theta(x) + D_{KL}(q_\phi(z|x)||p_\theta(z)) - \mathbb{E}_{z \sim q_\phi(z|x)}(\log p_\theta(x|z)) &\geq 0 \end{aligned}$$

So, we have,

$$\log p_\theta(x) \geq \mathbb{E}_{z \sim q_\phi(z|x)}(\log p_\theta(x|z)) - D_{KL}(q_\phi(z|x)||p_\theta(z)) \quad (4.10)$$

This inequality gives a lower bound on log evidence. This lower bound is called Evidence Lower Bound or ELBO [71]. ELBO is the same as the negative of the loss shown in **Equation 4.8**. So, maximising the ELBO is equivalent to minimizing the VAE loss.

4.2.6 Monte Carlo Estimate

Both of the term in **Equation 4.8** can be approximated with Monte Carlo Estimation [72]. If the KL divergence can not be solved analytically it can be approximated very well with Monte Carlo estimator. We could have done this by sampling n values $z_1, \dots, z_n$ from q , then evaluate the estimate as:

$$D_{KL}(q_\phi(z|x)||p_\theta(z)) = \frac{1}{n} \sum_{i=1}^n \log [q(z_i)] - \log [p(z_i)] \quad (4.11)$$

But in practice, sampling like this on the backward pass is not possible, as it is a stochastic process. In this way, the model can not be trained and it would not be able to learn. To remedy this a simple reparameterization trick is used.

4.2.7 Reparameterization Trick

To use Monte Carlo Estimation and still allow the model to learn, samples are taken, as $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ from an unit normal distribution instead of sampling directly from q_ϕ . We can convert this sample to our required one by scaling with mean, μ and shifting with standard deviation σ .

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \text{ where } \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

Here, $\odot$ denotes elementwise multiplication and $\mathbf{I}$ is the identity matrix. This is also shown in **Figure 4.5** and known as the reparameterization trick.

4.2.8 Attribute Vector

This section discusses the possibility with VAE without adding any discriminator or GAN architecture. This feature **was not used** in the final model.

There is an interesting feature of a Variational Autoencoder. This VAE model alone can somewhat work for a style transfer objective. After training, the decoder can work by itself, if a sample from the prior p_θ is given. The sample can be used to generate a random output. As mentioned earlier, the prior is not learned through training. It is kept fixed through the training and is chosen as flexible distribution like the multivariate gaussian.

We, can leverage this feature of VAE to serve our purpose, transferring the style of a given sentence. To do this, we first trained the model with our training dataset. Then we used the encoder to get the latent distribution of all our positive and negative sentences. We used these to calculate the mean positive vector, μ_{pos} and negative vector, μ_{neg} . This gave us our attribute vector:

$$\Delta = \mu_{\text{pos}} - \mu_{\text{neg}} \tag{4.12}$$

This can be used in a similar fashion like word embeddings and works somewhat like the example shown earlier, *King - (Man - Woman) = Queen*. We pass a positive or negative sentence

through the encoder to get positive latent vector, z_{pos} or negative latent vector, z_{neg} . Then we can transfer the style with,

$$\text{Negative}(z_{\text{pos}}) = \text{Decoder}(z_{\text{pos}} - \alpha \times \Delta) \quad (4.13)$$

$$\text{Positive}(z_{\text{neg}}) = \text{Decoder}(z_{\text{neg}} + \alpha \times \Delta) \quad (4.14)$$

α is a hyperparameter which can be tuned to vary the polarity margin. It can also be selected to best fit a scenario via cross validation [73].

4.2.9 Hybrid Structure with GAN

VAE alone did not suffice for the objective very well. It could reconstruct a sentence very well from the latent space. But transferring attributes with attribute vector was vaguely defined. It became difficult for the model to preserve both the context and polarity without explicitly learning about style transfer during training phase. Because of these complications we decided to introduce a discriminator, which can act as a classifier for our decoder and properly guide it to update its weights in a desired way.

To make this possible we concatenated latent vector z with a discrete value c with 0 or 1 denoting negative or positive respectively for sentence reconstruction. This value itself is sampled from a categorical distribution as a prior. This way, the updated VAE architecture tries to generate negative or positive sentence depending on the structured latent code c , while preserving the context of the original sentence from z . This updated VAE structure is shown in **Figure 4.6**.

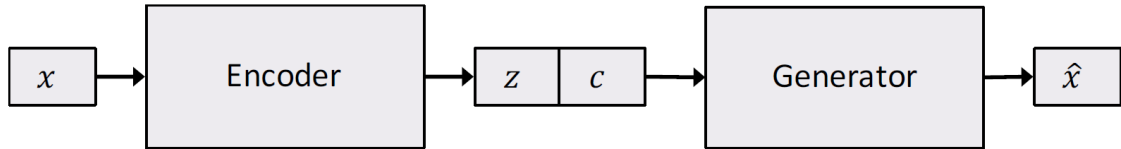


Figure 4.6: Updated VAE architecture with structured code c

The decoder of the VAE now works as a generator too for the GAN part. Complete architecture of the model is illustrated in **Figure 4.7**.

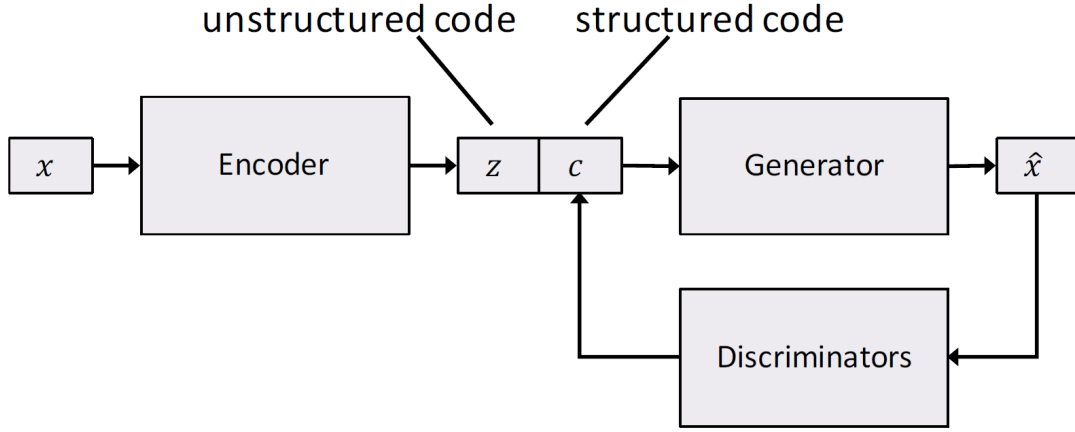


Figure 4.7: Hybrid Architecture with VAE and GAN

From here on, we will denote the Encoder, Decoder/Generator and Discriminator as E , G and D respectively. Their trainable parameters are denoted as ϕ , θ and σ respectively.

4.2.10 VAE Training

As, the generator is connected to both of the encoder and decoder and training all of them simultaneously is not possible, we train our model in a wake-sleep manner. The generator is trained in two separate ways.

First, the generator is trained to reconstruct the sentence when paired With the encoder. With the addition of structured code $\mathbf{c}$, we can rewrite **Equation 4.9** as:

$$\mathcal{L}_{\text{VAE}} = D_{KL}(q_{\phi}(z|x)||p_{\theta}(z)) - \mathbb{E}_{z \sim q_{\phi}(z|x), c \sim p_{\sigma}(c)}(\log p_{\theta}(x|z, c)) \quad (4.15)$$

Here, $p_{\sigma}(c)$ is the prior distribution of sentiments.

4.2.11 Generator Training

Equation 4.15 trains both of the encoder and the generator, but it does not look upon the sentiment of the reconstructed sentence. To catch the sentiment in reconstruction, generator is paired with the discriminator as shown in **Figure 4.8**.

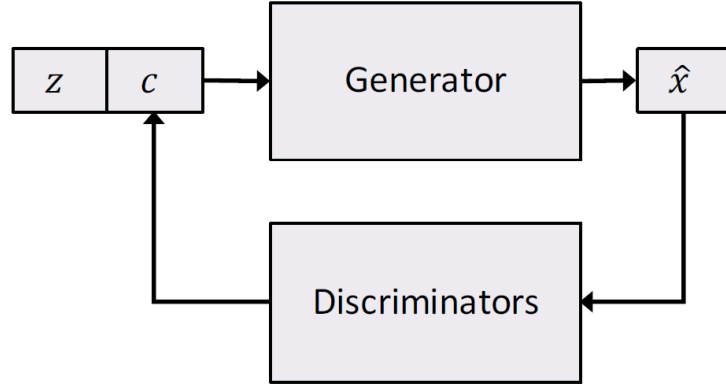


Figure 4.8: GAN part of the model

The generator can be trained to maximize the likelihood of given attribute on reconstructed sentence, which is shown in **Equation 4.16**.

$$\mathcal{L}_c = -\mathbb{E}_{z \sim p_\theta(z), c \sim p_\sigma(c)} (\log q_\sigma(c|\hat{x})) \quad (4.16)$$

Although, $\mathcal{L}_c$ makes sure that sentimental attribute is present in reconstruction from **c**, it might change the correlated but unwanted context structure from **z**. To make sure the generator is capturing the context properly from **z** another loss is added:

$$\mathcal{L}_z = -\mathbb{E}_{z \sim p_\theta(z), c \sim p_\sigma(c)} (\log q_\phi(z|\hat{x})) \quad (4.17)$$

Equation 4.16 and **4.17** together works as the loss of the generator during the exclusive learning of generator. So, the objective becomes:

$$\mathcal{L}_G = \lambda_c \mathcal{L}_c + \lambda_z \mathcal{L}_z \quad (4.18)$$

Here, λ_c and λ_z are hyperparameters and can be tuned. It is important to note that **Equation 4.18** trains only the generator weights, while encoder's and discriminator's weight are kept frozen. On the other hand, during the training of the Variational Autoencoder both of the encoder and the generator updates their weights using **Equation 4.15**.

4.2.12 Discriminator Training

The discriminator training is comparatively simple. It is first trained in a supervised setting to classify sentiments. It outputs a Bernoulli distribution depicting the probability of the sentiments. The loss is defined to maximize log likelihood of the correct sentiment.

$$\mathcal{L}_D = -\mathbb{E}_{x \sim X}(\log q_\sigma(c|x)) \quad (4.19)$$

Then, it is paired with the generator and uses the synthesised sentences produced by the generator to train itself. This makes the loss of the discriminator as:

$$\mathcal{L}_D = -\mathbb{E}_{x \sim X}(\log q_\sigma(c|x)) - \lambda_G \mathbb{E}_{\hat{x} \sim G(z,c)}(\log q_\sigma(c|\hat{x})) \quad (4.20)$$

Here, λ_G is a balancing hyperparameter.

4.2.13 Challenge in attribute learning

The decoder or generator produces probable tokens at each timestamps which is fed back into it to produce the next token. These tokens are discrete (represented as one-hot vectors) and can not be used to propagate gradient when paired with the discriminator. To remedy this problem, a softmax approximation with temperature control is used. Instead of using $\hat{\mathbf{x}}_t$ as a one-hot vectors, it is sampled from $\text{softmax}(\hat{\mathbf{x}}_t/\tau)$, where τ is the temperature. τ is set to 1 in the beginning and slowly shifted backward to 0, which can emulate the one-hot vector.

These approximations can be used easily to get an approximated word embedding by dot product. This embedding is used in the network to produce synthesised examples.

4.2.14 Training Overview

To summarize, the VAE is trained first with the full corpus of unlabeled data using **Equation 4.15**. Then the discriminator is trained using labeled data. Then repeated alternating training was followed, training the whole model until convergence.

Chapter 5

Implementation and Results

Deep learning frameworks have become very powerful in recent times. Our proposed model and its functionality can be implemented with the help of quite a few different frameworks like TensorFlow or PyTorch. Both of these framework are built on python. We decided to use TensorFlow along with the TensorFlow Probability library, which is also built on TensorFlow that makes it easy to combine probabilistic models and deep learning on modern hardware. We will describe the implementation details in this chapter.

5.1 Embedding Selection

We initially decided to use the trainable Embedding Layer included in TensorFlow library. But it did not produce good enough results for our perpose. Then, we decided to use transfer learning [74] and experimented with two different pretrained word embeddings on for Bengali language. Both are published by facebook using fasttext. They have embeddings available for 157 different languages which are available for mass use [75].

These are:

1. **Common Crawl Bengali:** This word embedding is trained on common crawl [76] and wikipedia. This is trained using Continuous Bag of Words model. The dimensions of the embedding vector is 300 and the total vocabulary length of the corpus is 1468578.

2. **Wikipedia Bengali:** This is an embedding trained on the Bengali Wikipedia articles. The embedding file is 375 Megabytes in size. It also has a embedding of length 300. It is trained using the skip-gram model. Total vocabulary length in it is 145350.

Both of these embeddings worked well with our purpose. As, our dataset contained crawled data, we kept the Common Crawl embedding as our final selection for the model as it had a much larger vocabulary which can catch uncommon words from our dataset.

5.2 Preprocessing

Several preprocessing steps were needed and it were equally effective if not more for the model's performance.

- We have used 4 different datasets and used the 10000 most frequent words as our vocabulary, including four extra tokens: **<PAD>**, **<SOS>**, **<EOS>**, **<UNK>**. These are tokens for *padding*, *start of sentence*, *end of sentence* and *out of vocabulary* words respectively.
- Different dataset were split differently for training set and test set based on the size of dataset. 10% of the training data were used in validation.
- All of our dataset contained inappropriate words and needed to be filtered. We tokenized the words using TensorFlow's tokenizer. Then, filtered out sentences with word length over 14. Sentences with lower than the specified max length are post padded with zeroes to have the same length.
- We used TensorFlow's builtin data pipeline and used 32 sentences in a batch for the model input.

5.3 Datasets

We explored a total of four datasets from different social media and news portals. These datasets were crawled and tagged with a decent accuracy. The datasets are:

1. Prothom-Alo comments
2. Socian AI dataset
3. Comments on Cricket [77]
4. Comments on Restaurants [77]

Prothom-Alo comments: This dataset is comprised with 13802 comments on the news portal Prothom-Alo. The comments are tagged in six different tags namely: 'Surely Negative', 'Slightly Negative', 'Neutral', 'Skip', 'Slightly Positive' and 'Surely Positive'. We used the mean value of these evaluation to split the dataset into positive and negative data.

Socian AI dataset was collected from a open source github repository [78], where only 4000 samples were hosted. This dataset was very well structured compared to the others. This is also a symmetric dataset with 2000 samples for each sentiment.

Comments on Cricket: Mass comments on cricket-related posts from different online portals were hosted in github. The comments are tagged positive and negative. There is a total of 2718 comments. This is a very biased dataset. The number of positive comments is 566 and negative is 2152.

Comments on Restaurants: From github, we got a dataset about comments on restaurant experiences. Positive, negative and neutral comments are tagged here. There are a total of 1693 comments. This is also a heavily biased dataset. 1221 comments are tagged as positive and 472 as negative.

We have initially tried training our model with different dataset where we have seen clear indication of supremacy of different datasets when comparing different attributes of the output. But moving on with any one of them was very difficult because of data size. Good reconstruction performance needed much more data than we had. So, we decided to merge all of our data for the final training.

5.4 Encoder

We structured the Encoder to embed the sentences in a 32 dimensional latent space. It contains 7 layers:

1. An **Embedding** layer with the pretrained embedding is used with embedding dimension of 300.
2. A **GRU** layers with 256 units and tanh activations, where all hidden state are forwarded for the next recurrent layer.
3. A **BatchNormalization** layer.
4. Another **GRU** layer with 128 units and tanh activations with just the last hidden state as output.
5. Another **BatchNormalization** layer.
6. A **Dense** layer with required units for the next probabilistic layer and linear activation.
7. A **MultivariateNormalTriL** layer is used from TensorFlow Probability. It uses the previous dense layer's outputs to generate a 32 dimensional latent space.

A strucrue is shown in **Figure 5.1**

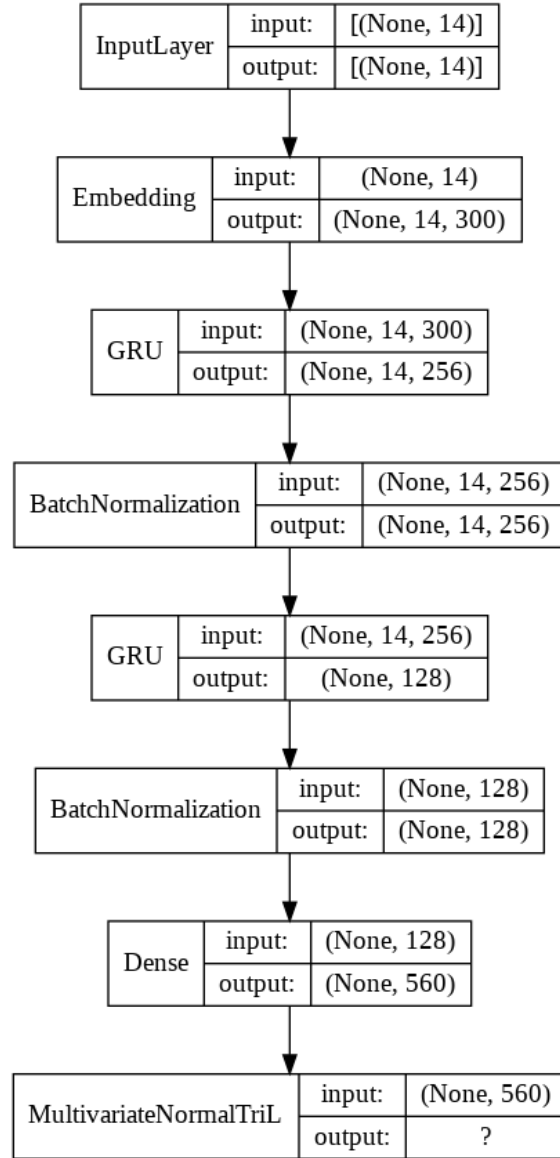


Figure 5.1: Encoder's structure

For the prior distribution, we used a mixture of 2 Gaussian distribution. It is fed into the **MultivariateNormalTriL** layer. KL divergence is calculated using Monte Carlo Estimation with 5 samples.

5.5 Generator

The generator was modelled using TensorFlow's model subclassing. It takes 2 inputs: a sample from the encoded latent space and actual input which also works as the output for reconstruction. But this input is shifted forward one timestamp and a <SOS> token is used as the first token. This decoder embeds the inputs first and uses latent sample as the initial state of GRU layer. A Bernoulli distribution was generated as output to account for the 10000 words in vocabulary. Decoder's structure is shown in **Figure 5.2**.

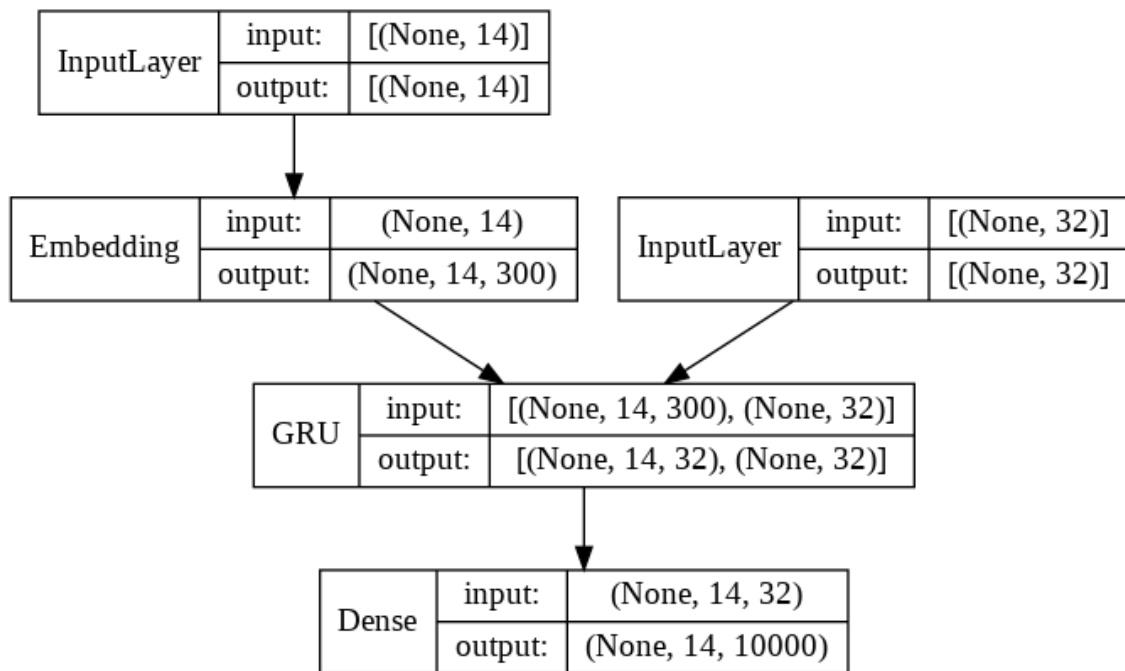


Figure 5.2: Decoder's structure

5.6 Discriminator

Discriminator was also created using model subclassing, as it needed different techniques during its different training phase. When paired with generator, the discriminator used a softmax activation on the inputs to approximate an embedding, which in turn allowed backpropagation of gradient from the discriminator to the generator. Aside this, discriminator's architecture was simple, with a binary classification problem at hand. This architecture is shown in **Figure 5.3**.

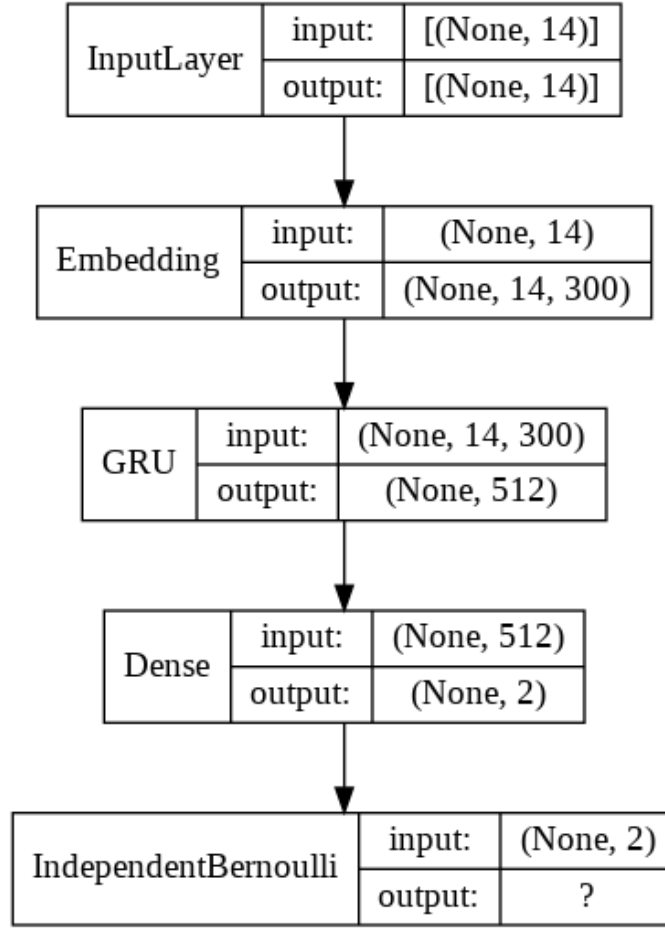


Figure 5.3: Discriminator's structure

Prior for the sentiment distribution was calculated based on the dataset. Final layer of the discriminator uses a Bernoulli distribution which is also used to take samples when training the VAE after initial training.

5.7 Training

We first trained the VAE model with all of During the training of VAE actual input was used as decoder inputs in each timestamp instead of the predicted ones, this is called Teacher forcing [79] and ensures a robust training for the model.

This initial training of the VAE was done on the merged dataset of all of the 4 datasets we had. As VAE's purpose was to reconstruct the input sentence without focusing on sentiment, large

corpus was very important.

After VAE's initial training, discriminator was trained separately on labeled data without connecting to the generator. The discriminator learned well to classify sentiments with 88% accuracy.

After this initial setup, alternating training of generator of discriminator and generator continued. The generator was trained in two phases:

- It was trained with the encoder with VAE loss from **Equation 4.15**. It focuses on the unstructured latent space z to preserve context.
- The generator learns to transfer polarity from discriminator's feedback. Keeping the discriminator's weight constant, generator was trained with **Equation 4.18**.

The discriminator is also trained in-between with samples from both real data and synthesised data from the generator. As the generator gets better at sentiment generation, discriminator's loss increases. We iteratively trained the generator and discriminator until we became satisfied with the outputs.

5.8 Evaluation

Our new model with discriminator showed much better content preservation than the previous VAE model with attribute vector. First we show some outputs from the model without discriminator in **Figure 5.4** and **5.5**.

```
[84] transfer_sentiment(vae, style='negative', "রাজজাকের পারফরমেন্স ভালো ছিলো")
```

রাজজাকের পারফরমেন্স ভালো ছিলো
Output: রাজজাক স্পিন খরাপ হচ্ছে দিনদিন ।

```
[86] transfer_sentiment(vae, style='negative', "সাকিব শেষের ছক্কাটা অসাধারণ ছিল।")
```

সাকিব শেষের ছক্কাটা অসাধারণ ছিল।
Output: আরে সব ভাঙতবাজি ।

```
▶ transfer_sentiment(vae, style='negative', "বিপিএলে রাজশাহী ভাল খেলেছে।")
```

❏ বিপিএলে রাজশাহী ভাল খেলেছে।
Output: বিপিএলে উইকেট পড়ছে পড়ছে ।

Figure 5.4: Positive to Negative Style Transfer without discriminator

```
[88] transfer_sentiment(vae, style='positive', "সাকিব অনেক বাজে খেলেছে।")
```

সাকিব অনেক বাজে খেলেছে।
Output: তবুও জয়ের আশা ছাড়ছি না ।

```
[89] transfer_sentiment(vae, style='positive', "এরচেয়ে বাজে খেলা যায় না।")
```

এরচেয়ে বাজে খেলা যায় না।
Output: বাংলাদেশের খেলার ব্যাটসম্যান প্রেয়ার ।

```
▶ transfer_sentiment(vae, style='positive', "তামিম আজকে তেমন ভাল খেলেনি।")
```

❏ তামিম আজকে তেমন ভাল খেলেনি।
Output: আমার প্রিয় খেলোয়াড় , , অনেক দিন পরে দেখলাম ।

Figure 5.5: Negative to Positive Style Transfer without discriminator

The following predictions shown in **Figure 5.6** and **5.7** are sampled from our final model on the same input sentences.

```
[77] transfer_style('রাজজাকের পারফরমেন্স ভালো ছিল।', style_to='negative')
```

```
input: রাজজাকের পারফরমেন্স ভালো ছিল।  
output: রাজজাকের সাথে বেঈমানী করেছে বিসিবি ।
```

```
[78] transfer_style('সাকিব শেষের ছক্কাটি অসাধারণ ছিল।', style_to='negative')
```

```
input: সাকিব শেষের ছক্কাটি অসাধারণ ছিল।  
output: সাকিব শেষের ছক্কাটি <UNK> <UNK> ।
```

```
[79] transfer_style('বিপিএলে রাজশাহী ভাল খেলেছে।', style_to='negative')
```

```
input: বিপিএলে রাজশাহী ভাল খেলেছে।  
output: রাজশাহী খেলেছে । ভাষা হারিয়ে ফেলেছি।
```

Figure 5.6: Positive to Negative Style Transfer with discriminator

```
[107] transfer_style('সাকিব অনেক বাজে খেলেছে।', style_to='positive')
```

```
input: সাকিব অনেক বাজে খেলেছে।  
output: সাকিব বিশ্বসেরা অলরাউন্ডার ।
```

```
[108] transfer_style('এরচেয়ে বাজে খেলা যায় না।', style_to='positive')
```

```
input: এরচেয়ে বাজে খেলা যায় না।  
output: আরও ভাল খেলা যায় ।
```

```
[109] transfer_style('তামিম আজকে তেমন ভাল খেলেনি।', style_to='positive')
```

```
input: তামিম আজকে তেমন ভাল খেলেনি।  
output: তামিম ভাল খেলেছে , , দোয়া রইল ।
```

Figure 5.7: Negative to Positive Style Transfer with discriminator

5.8.0.1 Human Evaluation

Evaluating the performance of our model with metrics was very hard as it had many different features, that it can be evaluated from very different perspectives. We decided to human evaluate these model's performance by sampling predictions of the 3 different models with 30 input samples. These three models are:

- VAE with multi-decoders
- VAE with attribute vectors
- VAE with discriminator

We asked the volunteers to rate the outputs on three different metrics: Context Preservation, Grammar and Polarity Transfer. Each metric was evaluated on a scale of 0 to 10. We have received feedback from 17 volunteers. Average score on each metric for the models are shown in the table below:

Score on a scale of 10			
Models	Context	Grammar	Polarity
VAE with multi-decoders	4.6	6.3	7.9
VAE with attribute vectors	6.3	4.8	7.2
VAE with discriminator	8.3	6.4	7.4

Table 5.1: Human evaluation on different metrics for different models

We can see that our final model improves further on context preservation compared to the previous one. It also transfers sentiment quite well and has moderate grammatical structure. Grammatical structure is somewhat lacking because of the dataset quality. We expect the grammar to be much better when trained on a better dataset.

Chapter 6

Conclusion

Field of Natural Language Processing has reached new milestones in recent years which makes previous state of the art models look almost naive. With gigantic amount of language corpora, computation power to process these giants have become the difference between a good and bad model. With monumental success in this area, few scopes are left for theoretical improvements.

While exploring this vast field, we found that variational and probabilistic methods for deep learning are largely unexplored as they are more complicated to understand and train properly. Our approach included a study on different discriminative and generative methods. With sentimental style transfer of text our study explores only a glimpse of probabilistic deep neural networks and shows how powerful they can be.

As there has been no significant work on this domain, we struggled to find a proper solution for our purpose. We have experimented with different model architectures and finally proposed a hybrid model architecture of VAE and GAN. Our model shows significant improvements over the previous ones, as it has its learning objectives of reconstructing sentence and attribute changing separated. With the help of VAE it shows very good content preservation and discriminator helps it to include desired sentiment in the reconstruction process.

Alongside the original objective it was trained on, our model also offers some versatile options which can be used for different objectives. The VAE architecture can alone be used for sentence reconstruction for purpose like paraphrasing. It can also be used to generate random sentences by sampling from the prior distribution and feeding them in the generator.

The structured code added to the latent space also has some possibilities. As the attributes are added separately, our model shows the possibility to add multiple structured code instead of just one. This can also be a continuous value rather than the discrete one we had. This can be achieved by adding multiple discriminators for the different attributes and modifying them accordingly. For example, continuous attributes can be modeled with discriminators trying to fit regression line instead of acting as a classifier.

References

- [1] G.-F. Peng, Y.-S. Yang, C.-Y. Tsai, and V.-W. Soo, “Generate modern chinese poems from news based on text style transfer using gan,” in *2019 International Conference on Technologies and Applications of Artificial Intelligence (TAAI)*. IEEE, 2019, pp. 1–6.
- [2] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” 2014.
- [3] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” *Advances in neural information processing systems*, vol. 27, 2014.
- [4] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired image-to-image translation using cycle-consistent adversarial networks,” 2020.
- [5] M. H. Palash, P. P. Das, and S. Haque, “Sentimental style transfer in text with multigenerative variational auto-encoder,” in *2019 International Conference on Bangla Speech and Language Processing (ICBSLP)*. IEEE, 2019, pp. 1–4.
- [6] A. M. TURING, “I. COMPUTING MACHINERY AND INTELLIGENCE,” *Mind*, vol. LIX, no. 236, pp. 433–460, 10 1950. [Online]. Available: <https://doi.org/10.1093/mind/LIX.236.433>
- [7] J. Y. Yam and T. W. Chow, “A weight initialization method for improving training speed in feedforward neural network,” *Neurocomputing*, vol. 30, no. 1-4, pp. 219–232, 2000.
- [8] S. K. Kumar, “On weight initialization in deep neural networks,” *arXiv preprint arXiv:1704.08863*, 2017.

- [9] A. Pavelka and A. Procházka, “Algorithms for initialization of neural network weights,” in *In Proceedings of the 12th Annual Conference, MATLAB*, 2004, pp. 453–459.
- [10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [11] L. Bottou, “Large-scale machine learning with stochastic gradient descent,” in *Proceedings of COMPSTAT’2010*. Springer, 2010, pp. 177–186.
- [12] T. Kurbiel and S. Khaleghian, “Training of deep neural networks based on distance measures using rmsprop,” *arXiv preprint arXiv:1708.01911*, 2017.
- [13] Z. Zhang, “Improved adam optimizer for deep neural networks,” in *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*. IEEE, 2018, pp. 1–2.
- [14] A. Gotmare, N. S. Keskar, C. Xiong, and R. Socher, “A closer look at deep learning heuristics: Learning rate restarts, warmup and distillation,” *arXiv preprint arXiv:1810.13243*, 2018.
- [15] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, “How does batch normalization help optimization?” *arXiv preprint arXiv:1805.11604*, 2018.
- [16] X. Ying, “An overview of overfitting and its solutions,” in *Journal of Physics: Conference Series*, vol. 1168, no. 2. IOP Publishing, 2019, p. 022022.
- [17] J. Byrd and Z. Lipton, “What is the effect of importance weighting in deep learning?” in *International Conference on Machine Learning*. PMLR, 2019, pp. 872–881.
- [18] T. Van Laarhoven, “L2 regularization versus batch and weight normalization,” *arXiv preprint arXiv:1706.05350*, 2017.
- [19] C. Garbin, X. Zhu, and O. Marques, “Dropout vs. batch normalization: an empirical study of their impact to deep learning,” *Multimedia Tools and Applications*, pp. 1–39, 2020.
- [20] Y. Gal, J. Hron, and A. Kendall, “Concrete dropout,” *arXiv preprint arXiv:1705.07832*, 2017.
- [21] S. Albawi, T. A. Mohammed, and S. Al-Zawi, “Understanding of a convolutional neural network,” in *2017 International Conference on Engineering and Technology (ICET)*. Ieee, 2017, pp. 1–6.

- [22] A. Sherstinsky, “Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network,” *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, 2020.
- [23] W. Yin, K. Kann, M. Yu, and H. Schütze, “Comparative study of cnn and rnn for natural language processing,” *arXiv preprint arXiv:1702.01923*, 2017.
- [24] S. Sharma and S. Sharma, “Activation functions in neural networks,” *Towards Data Science*, vol. 6, no. 12, pp. 310–316, 2017.
- [25] G. Philipp, D. Song, and J. G. Carbonell, “The exploding gradient problem demystified-definition, prevalence, impact, origin, tradeoffs, and solutions,” *arXiv preprint arXiv:1712.05577*, 2017.
- [26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *arXiv preprint arXiv:1706.03762*, 2017.
- [27] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” 2014.
- [28] K. Han, Y. Wang, C. Zhang, C. Li, and C. Xu, “Autoencoder inspired unsupervised feature selection,” in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2018, pp. 2941–2945.
- [29] L. Theis, W. Shi, A. Cunningham, and F. Huszár, “Lossy image compression with compressive autoencoders,” *arXiv preprint arXiv:1703.00395*, 2017.
- [30] M. Nishio, C. Nagashima, S. Hirabayashi, A. Ohnishi, K. Sasaki, T. Sagawa, M. Hamada, and T. Yamashita, “Convolutional auto-encoder for image denoising of ultra-low-dose ct,” *Heliyon*, vol. 3, no. 8, p. e00393, 2017.
- [31] R. Senge, S. Bösner, K. Dembczyński, J. Haasenritter, O. Hirsch, N. Donner-Banzhoff, and E. Hüllermeier, “Reliable classification: Learning classifiers that distinguish aleatoric and epistemic uncertainty,” *Information Sciences*, vol. 255, pp. 16–29, 2014.

- [32] K. Chowdhary and P. Dupuis, “Distinguishing and integrating aleatoric and epistemic variation in uncertainty quantification,” *ESAIM: Mathematical Modelling and Numerical Analysis-Modélisation Mathématique et Analyse Numérique*, vol. 47, no. 3, pp. 635–662, 2013.
- [33] G. E. Hinton, “Boltzmann machine,” *Scholarpedia*, vol. 2, no. 5, p. 1668, 2007.
- [34] P. Baldi, “Autoencoders, unsupervised learning, and deep architectures,” in *Proceedings of ICML workshop on unsupervised and transfer learning*. JMLR Workshop and Conference Proceedings, 2012, pp. 37–49.
- [35] A. F. Veinott Jr, “Markov decision chains.” STANFORD UNIV CALIF DEPT OF OPERATIONS RESEARCH, Tech. Rep., 1973.
- [36] Y. Viazovetskyi, V. Ivashkin, and E. Kashin, “Stylegan2 distillation for feed-forward image manipulation,” in *European Conference on Computer Vision*. Springer, 2020, pp. 170–186.
- [37] T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, “Analyzing and improving the image quality of stylegan,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 8110–8119.
- [38] [Online]. Available: <https://thispersondoesnotexist.com/>
- [39] M. Mirza and S. Osindero, “Conditional generative adversarial nets,” 2014.
- [40] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired image-to-image translation using cycle-consistent adversarial networks,” in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2223–2232.
- [41] C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, J. Totz, Z. Wang *et al.*, “Photo-realistic single image super-resolution using a generative adversarial network,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4681–4690.
- [42] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” 2018.

- [43] R. A. Yeh, C. Chen, T. Yian Lim, A. G. Schwing, M. Hasegawa-Johnson, and M. N. Do, "Semantic image inpainting with deep generative models," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 5485–5493.
- [44] H. Zhang, T. Xu, H. Li, S. Zhang, X. Wang, X. Huang, and D. Metaxas, "Stackgan: Text to photo-realistic image synthesis with stacked generative adversarial networks," 2017.
- [45] S. Yang, L. Xie, X. Chen, X. Lou, X. Zhu, D. Huang, and H. Li, "Statistical parametric speech synthesis using generative adversarial networks under a multi-task learning framework," in *2017 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2017, pp. 685–691.
- [46] J. Mueller, D. Gifford, and T. Jaakkola, "Sequence to better sequence: Continuous revision of combinatorial structures," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 2536–2544. [Online]. Available: <http://proceedings.mlr.press/v70/mueller17a.html>
- [47] I. P. Yamshchikov, V. Shibaev, A. Nagaev, J. Jost, and A. Tikhonov, "Decomposing textual information for style transfer," *arXiv preprint arXiv:1909.12928*, 2019.
- [48] Z. Hu, Z. Yang, X. Liang, R. Salakhutdinov, and E. P. Xing, "Toward controlled generation of text," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 1587–1596. [Online]. Available: <http://proceedings.mlr.press/v70/hu17e.html>
- [49] J. Li, R. Jia, H. He, and P. Liang, "Delete, retrieve, generate: A simple approach to sentiment and style transfer," *arXiv preprint arXiv:1804.06437*, 2018.
- [50] A. Singh and R. Palod, "Sentiment transfer using seq2seq adversarial autoencoders," *arXiv preprint arXiv:1804.04003*, 2018.

- [51] J. Xu, X. Sun, Q. Zeng, X. Ren, X. Zhang, H. Wang, and W. Li, “Unpaired sentiment-to-sentiment translation: A cycled reinforcement learning approach,” *arXiv preprint arXiv:1805.05181*, 2018.
- [52] L. Yi, D. Li, P. Li, S. Shi, W. Lam, and T. Zhang, “Learning sentiment memories for sentiment modification without parallel data,” in *Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 3855–3864.
- [53] Y. Cheng, Z. Gan, Y. Zhang, O. Elachqar, D. Li, and J. Liu, “Contextual text style transfer,” *arXiv preprint arXiv:2005.00136*, 2020.
- [54] D. Li, Y. Zhang, Z. Gan, Y. Cheng, C. Brockett, M.-T. Sun, and B. Dolan, “Domain adaptive text style transfer,” *arXiv preprint arXiv:1908.09395*, 2019.
- [55] Z. Jin, D. Jin, J. Mueller, N. Matthews, and E. Santus, “Imat: Unsupervised text attribute transfer via iterative matching and translation,” *arXiv preprint arXiv:1901.11333*, 2019.
- [56] K. Carlson, A. Riddell, and D. Rockmore, “Evaluating prose style transfer with the bible,” *Royal Society open science*, vol. 5, no. 10, p. 171920, 2018.
- [57] Y. Wang, Y. Wu, L. Mou, Z. Li, and W. Chao, “Formality style transfer with shared latent space,” in *Proceedings of the 28th International Conference on Computational Linguistics*, 2020, pp. 2236–2249.
- [58] —, “Harnessing pre-trained neural networks with rules for formality style transfer,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3573–3578. [Online]. Available: <https://www.aclweb.org/anthology/D19-1365>
- [59] A. Gatt and E. Krahmer, “Survey of the state of the art in natural language generation: Core tasks, applications and evaluation,” *Journal of Artificial Intelligence Research*, vol. 61, pp. 65–170, 2018.
- [60] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” 2014.

- [61] M. X. Chen, O. Firat, A. Bapna, M. Johnson, W. Macherey, G. Foster, L. Jones, N. Parmar, M. Schuster, Z. Chen *et al.*, “The best of both worlds: Combining recent advances in neural machine translation,” *arXiv preprint arXiv:1804.09849*, 2018.
- [62] M. Kusner, Y. Sun, N. Kolkin, and K. Weinberger, “From word embeddings to document distances,” in *International conference on machine learning*. PMLR, 2015, pp. 957–966.
- [63] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *arXiv preprint arXiv:2005.14165*, 2020.
- [64] Wikipedia, “One-hot, Wikipedia,” <http://en.wikipedia.org/w/index.php?title=One-hot&oldid=1018673079>, 2021, [Online; accessed 16-June-2021].
- [65] J. Pennington, R. Socher, and C. D. Manning, “Glove: Global vectors for word representation,” in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1532–1543.
- [66] S. Balakrishnama and A. Ganapathiraju, “Linear discriminant analysis-a brief tutorial,” *Institute for Signal and information Processing*, vol. 18, no. 1998, pp. 1–8, 1998.
- [67] J. Joyce, “Bayes’ theorem,” 2003.
- [68] Wikipedia, “Variational Bayesian methods — Wikipedia, the free encyclopedia,” <http://en.wikipedia.org/w/index.php?title=Variational%20Bayesian%20methods&oldid=1025315825>, 2021, [Online; accessed 16-June-2021].
- [69] —, “Kullback Leibler divergence — Wikipedia, the free encyclopedia,” <http://en.wikipedia.org/w/index.php?title=Kullback%E2%80%93Leibler%20divergence&oldid=1026917999>, 2021, [Online; accessed 16-June-2021].
- [70] —, “Gibbs’ inequality — Wikipedia, the free encyclopedia,” <http://en.wikipedia.org/w/index.php?title=Gibbs’%20inequality&oldid=1012707845>, 2021, [Online; accessed 16-June-2021].

- [71] —, “Evidence lower bound — Wikipedia, the free encyclopedia,” <http://en.wikipedia.org/w/index.php?title=Evidence%20lower%20bound&oldid=1021680760>, 2021, [Online; accessed 16-June-2021].
- [72] P. Dagum, R. Karp, M. Luby, and S. Ross, “An optimal algorithm for monte carlo estimation,” *SIAM Journal on computing*, vol. 29, no. 5, pp. 1484–1496, 2000.
- [73] M. W. Browne, “Cross-validation methods,” *Journal of mathematical psychology*, vol. 44, no. 1, pp. 108–132, 2000.
- [74] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Transactions on knowledge and data engineering*, vol. 22, no. 10, pp. 1345–1359, 2009.
- [75] “Word vectors for 157 languages — fasttext.” [Online]. Available: <https://fasttext.cc/docs/en/crawl-vectors.html>
- [76] [Online]. Available: <https://commoncrawl.org/>
- [77] AtikRahman, “Atikrahman/bangla_absa_datasets.” [Online]. Available: https://github.com/AtikRahman/Bangla_ABSA_Datasets
- [78] Socian-Ai, “socian-ai/socian-bangla-sentiment-dataset-labeled.” [Online]. Available: <https://github.com/socian-ai/socian-bangla-sentiment-dataset-labeled>
- [79] A. Lamb, A. Goyal, Y. Zhang, S. Zhang, A. Courville, and Y. Bengio, “Professor forcing: A new algorithm for training recurrent networks,” *arXiv preprint arXiv:1610.09038*, 2016.